

Universidad ORT Uruguay
Facultad de Ingeniería

Interfaz conversacional para la planificación de estudios

Entregado como requisito para la obtención del
título de Licenciatura en Sistemas

Nicolás Duarte - 268772
Mateo Gauthier - 217385
Felipe Rodríguez - 258250

Tutor: Daniel Calegari

2025

Nosotros, Mateo Gauthier, Nicolás Duarte y Felipe Rodríguez declaramos que el trabajo que se presenta en esta obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el Proyecto Final de Licenciatura en Sistemas;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra;
- En la obra, hemos acusado recibo de las ayudas recibidas;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros y qué fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.



Nicolás Duarte



Mateo Gauthier



Felipe Rodríguez

{15-09-2025}

Dedicatoria

Dedicamos este trabajo a nuestras familias, por el apoyo y acompañamiento brindado a lo largo de toda la carrera, y por motivarnos a alcanzar este objetivo.

También lo dedicamos a quienes compartieron este proceso con nosotros, tanto dentro como fuera del ámbito académico, aportando su apoyo durante las distintas etapas de nuestra formación.

Agradecimientos

Agradecemos especialmente a nuestro tutor, Daniel Calegari, por su orientación, disposición y acompañamiento durante el desarrollo de este proyecto.

Asimismo, agradecemos a los docentes de la carrera por los conocimientos brindados a lo largo de nuestra formación y a todas las personas que, de una u otra manera, contribuyeron a la realización de este trabajo.

Abstract

Este proyecto presenta el desarrollo de una interfaz conversacional para la planificación de estudios en el ámbito de la educación superior. La solución propuesta utiliza técnicas de Minería de Datos Educativos (EDM) y una arquitectura de Generación Aumentada por Recuperación (RAG) para proporcionar asistencia personalizada a los estudiantes mediante consultas en lenguaje natural.

El sistema integra modelos de lenguaje de gran tamaño (LLMs) de código abierto, ejecutados localmente a través de Ollama, una base de datos vectorial (ChromaDB) para almacenar embeddings de documentos institucionales y un backend desarrollado en FastAPI que orquesta el flujo RAG mediante LangChain. El frontend, construido en React, ofrece una interfaz intuitiva que permite a los estudiantes consultar su trayectoria académica y sus planes de estudio y recibir recomendaciones personalizadas.

La principal contribución de este trabajo es demostrar la viabilidad de una solución de asistencia académica basada en IA que mantiene la privacidad de los datos mediante la ejecución local de modelos, evita alucinaciones mediante RAG y proporciona respuestas fundamentadas en información institucional verificable. Los resultados muestran que el sistema es capaz de responder con precisión consultas complejas sobre planificación académica, utilizando información extraída de documentos institucionales y de trayectorias estudiantiles históricas.

La implementación se realizó como una prueba de concepto funcional, validando tanto la arquitectura técnica como la utilidad práctica de la interfaz conversacional en el contexto universitario. El sistema desarrollado sienta las bases para futuras mejoras e integraciones con sistemas de gestión académica existentes.

Palabras clave

Machine Learning Campo de la inteligencia artificial que consiste en entrenar algoritmos para reconocer patrones y realizar predicciones a partir de datos.

Clustering Algoritmo de machine learning no supervisado que organiza y clasifica diferentes objetos, puntos de datos u observaciones en grupos o clústeres basados en similitudes o patrones.

Chatbot Es un programa de software que simula una conversación humana, ya sea por texto o por voz.

LLM (Large Language Model) Modelos de lenguaje entrenados con grandes volúmenes de texto para comprender y generar lenguaje natural.

Token objeto o dato digital que representa algo de valor. En el contexto de los LLM, se refieren a palabras, juegos de caracteres o combinaciones de palabras y puntuación que generan los modelos al descomponer el texto.

Ollama Entorno de ejecución local para modelos de lenguaje, que permite descargarlos y ejecutarlos en la propia máquina.

DeepSeek Proyecto y familia de modelos de lenguaje abiertos enfocados en el razonamiento eficiente y el rendimiento competitivo.

Qwen Familia de grandes modelos de lenguaje (LLM) desarrollados por Alibaba Cloud que pueden comprender y generar texto, así como procesar datos multimodales como imágenes y código.

ChatGPT Asistente conversacional basado en modelos de lenguaje desarrollado por OpenAI, diseñado para interactuar en lenguaje natural.

RAG (Retrieval-Augmented Generation) Técnica que combina la recuperación de información en bases de datos con la generación de texto mediante un modelo de lenguaje.

LangChain Framework en Python para desarrollar aplicaciones que integran modelos de lenguaje con memorias, cadenas de prompts y bases de conocimiento.

Prompt Instrucción o pregunta proporcionada a un modelo de lenguaje como entrada para obtener una respuesta específica.

Uvicorn Servidor ASGI rápido y ligero para Python, usado con frecuencia en aplicaciones web con frameworks como FastAPI.

ChromaDB Base de datos vectorial optimizada para almacenar y recuperar embeddings, comúnmente utilizada en aplicaciones de RAG.

API Conjunto de reglas y protocolos que permite que dos aplicaciones de software se comuniquen entre sí para intercambiar datos y funcionalidades.

Máquinas virtuales Emulación de un ordenador físico basada en software que puede ejecutar su propio sistema operativo y aplicaciones.

Malware Software malicioso diseñado para dañar, interrumpir o obtener acceso no autorizado a sistemas informáticos.

Índice general

1. Introducción	10
2. Marco teórico	12
2.1. Inteligencia Artificial	12
2.1.1. Machine Learning	13
2.2. Minería de Datos Educativos	14
2.2.1. Análisis Curricular	16
2.2.2. Proyecto de referencia: IA Aplicada a la Planificación de Estudios	17
2.3. Interfaces Conversacionales	19
2.3.1. Large Language Models (LLMs)	20
2.3.2. Generación Aumentada por Recuperación (RAG)	24
2.3.3. De RAG a Agentes Inteligentes	30
2.4. Frameworks de desarrollo	30
2.4.1. FastAPI	31
2.4.2. Uvicorn	32
2.4.3. Node.js	32
2.4.4. Flask	33
2.4.5. Django	33
2.5. Plataformas de despliegue	34
2.5.1. Servidores	34
2.5.2. Empaquetado, aislamiento y ejecución de la aplicación	39
2.6. Síntesis y Conexión con el Proyecto	41
3. Planteamiento del problema	43
3.1. Contexto	43
3.2. Requisitos	44
3.2.1. Requisitos funcionales	44
3.2.2. Requisitos no funcionales	47

4. Diseño e implementación de la solución	51
4.1. Metodología de trabajo adoptada	51
4.2. Etapas del desarrollo	52
4.3. Solución del problema	53
4.3.1. Backend	54
4.3.2. Frontend	67
4.3.3. DevOps	68
4.3.4. Comparativa tecnológica	69
4.4. Evaluación de la solución	71
4.4.1. Estrategia de evaluación	71
4.4.2. Resultados de las pruebas	72
5. Caso de aplicación	74
5.1. Ventanas del sitio	74
5.1.1. Página de bienvenida (Landing)	75
5.1.2. Inicio de sesión (Auth0)	76
5.1.3. Perfil del estudiante	77
5.1.4. Mis Cursos: progreso académico	78
5.1.5. Currícula de la carrera	79
5.1.6. Plan de estudios	80
5.1.7. Configuración del sistema	80
5.1.8. Estadísticas del sistema (administrador)	82
5.2. Consulta sobre materias aprobadas	83
5.3. Consulta de materias disponibles para inscripción	84
5.4. Manejo de archivos	87
5.5. Feedback de estudiantes	88
5.6. Recomendación de materias a cursar	91
6. Discusión y Limitaciones	92
6.1. Discusión del caso de aplicación	92
6.2. Limitaciones técnicas	93
6.2.1. Limitaciones de recursos y costos	93
6.2.2. Limitaciones de los modelos locales	94
6.2.3. Limitaciones del agente en acciones	94
6.3. Aspectos éticos de la investigación	95
6.3.1. Privacidad y protección de datos institucionales y personales	95
6.3.2. Riesgos de alucinaciones y legitimidad de la información	96
6.3.3. Sostenibilidad e incertidumbre de la industria de los LLMs	97
6.3.4. Análisis de costos y viabilidad económica del hosting	97

7. Conclusiones	99
7.1. Principales resultados	99
7.2. Aplicación de los resultados	100
7.3. Limitaciones	100
7.4. Líneas de trabajo futuro	101
8. Bibliografía	102

1 Introducción

El presente proyecto de investigación se enmarca en el campo de la Minería de Datos Educativos (EDM, por sus siglas en inglés *Educational Data Mining*), disciplina que aplica técnicas de inteligencia artificial, aprendizaje automático y ciencia de datos para analizar información generada en contextos educativos [1]. Su propósito es descubrir patrones y conocimientos útiles que permitan mejorar los procesos de enseñanza, aprendizaje y gestión académica.

En la actualidad, las instituciones de educación superior generan una gran cantidad de datos sobre el desempeño académico, la inscripción en cursos y las trayectorias de los estudiantes. Sin embargo, estos datos a menudo no se aprovechan plenamente para ofrecer apoyo personalizado ni facilitar la toma de decisiones académicas. En este contexto, las interfaces conversacionales —sistemas de software que permiten la interacción mediante lenguaje natural [2]— pueden constituir un medio intuitivo y accesible para que los estudiantes consulten y aprovechen esta información. Estas interfaces, impulsadas por los avances recientes en Modelos de Lenguaje de Gran Tamaño (LLMs, *Large Language Models*) [3] y técnicas de Generación Aumentada por Recuperación (RAG, *Retrieval-Augmented Generation*) [4], permiten generar respuestas fundamentadas en datos institucionales concretos, reduciendo el riesgo de alucinaciones y ofreciendo información verificable.

Una aplicación clave de la EDM es brindar asistencia a los estudiantes en la planificación de sus estudios. Este proyecto busca precisamente desarrollar una herramienta que facilite esta tarea. La propuesta se centra en analizar, diseñar y desarrollar una prueba de concepto de una **interfaz conversacional** que se integre con una herramienta de planificación de estudios. La finalidad es ofrecer a los estudiantes respuestas personalizadas sobre su carrera y su situación académica y generar recomendaciones basadas en sus intereses, como las materias a cursar el siguiente semestre.

Los objetivos específicos son:

1. Analizar alternativas tecnológicas para la implementación de interfaces conversacionales.
2. Desarrollar una prueba de concepto de la interfaz integrada a una herramienta de planificación (disponible en [5]).
3. Evaluar sus fortalezas y debilidades mediante un caso de aplicación concreto en la propia universidad.

El resto del documento se organiza de la siguiente manera. En el Capítulo 2 se presenta el marco teórico, abordando los conceptos fundamentales de inteligencia artificial, aprendizaje automático, EDM, interfaces conversacionales y las tecnologías empleadas, así como los proyectos relacionados. En el Capítulo 3 se describe el planteamiento del problema, incluyendo el contexto, los requisitos funcionales y no funcionales y su validación. El Capítulo 4 detalla el diseño e implementación de la solución, cubriendo la metodología de trabajo, la arquitectura del backend y del frontend, y la evaluación realizada. En el Capítulo 5 se presenta el caso de aplicación concreto en la universidad. El Capítulo 6 discute los resultados obtenidos, las limitaciones técnicas y los aspectos éticos de la investigación. Finalmente, en el Capítulo 7 se presentan las conclusiones y las líneas de trabajo futuras.

2 Marco teórico

En este capítulo se presentan los conceptos teóricos fundamentales que sustentan el desarrollo del proyecto. Se comienza con una introducción a la inteligencia artificial y sus subcampos más relevantes —el aprendizaje automático, el aprendizaje profundo y la inteligencia artificial generativa—, para luego abordar la minería de datos educativos y el análisis curricular como disciplinas aplicadas al contexto de este trabajo. Finalmente, se exploran las tecnologías clave para la solución propuesta: las interfaces conversacionales, los modelos de lenguaje de gran tamaño (LLMs) y las arquitecturas de recuperación de información.

2.1 Inteligencia Artificial

La Inteligencia Artificial (IA) puede entenderse, en un sentido amplio, como el campo interdisciplinario de la informática que se encarga de desarrollar sistemas capaces de realizar tareas que, de ser ejecutadas por personas, requerirían inteligencia. Estas tareas van desde el reconocimiento de patrones, la toma de decisiones, el aprendizaje a partir de datos hasta la resolución de problemas complejos [6].

Se puede entender el origen de la misma en la década de 1950, cuando surgió la posibilidad de reproducir en máquinas el razonamiento humano, en la prueba de Turing y luego formalizada en la conferencia de Dartmouth de 1956, donde se propuso que la inteligencia podía ser descrita y simulada computacionalmente [6] [7]. Desde entonces, el campo evolucionó hacia una enorme variedad de enfoques y niveles de autonomía, que van desde sistemas expertos y algoritmos de aprendizaje automático hasta modelos de aprendizaje profundo con capacidades predictivas avanzadas.

Desde un punto de vista técnico, la IA se basa en la capacidad de los algoritmos para aprender a partir de los datos, identificar patrones y generar predicciones o recomendaciones de manera autónoma. Esto se puede ver en técnicas como el ma-

chine learning, donde los sistemas mejoran su desempeño a partir de la experiencia, y el deep learning, que utiliza redes neuronales artificiales inspiradas en la estructura del cerebro humano [7]. En este sentido, la IA no se limita a la automatización de tareas, sino que introduce una nueva forma de procesamiento de información, caracterizada por su capacidad adaptativa y su potencial para transformar sectores como la medicina, la educación, la industria o las finanzas [7] [8].

Como se ilustra en la Figura 2.1, la IA abarca una jerarquía de subcampos con niveles crecientes de especialización. El **Machine Learning** (ML) es un subcampo de la IA que permite a los sistemas aprender patrones a partir de datos sin ser programados explícitamente. En el ML, el **Deep Learning** (DL) utiliza redes neuronales profundas con múltiples capas para modelar representaciones complejas de los datos. Finalmente, la **Inteligencia Artificial Generativa** (GenAI) representa la frontera más reciente, en la que modelos como los LLMs son capaces de generar texto, código, imágenes y otros contenidos a partir de los patrones aprendidos durante su entrenamiento. Esta jerarquía es relevante para el presente proyecto, ya que la solución propuesta se apoya en modelos de IA generativa (LLMs) que, a su vez, se basan en técnicas de deep learning y machine learning.

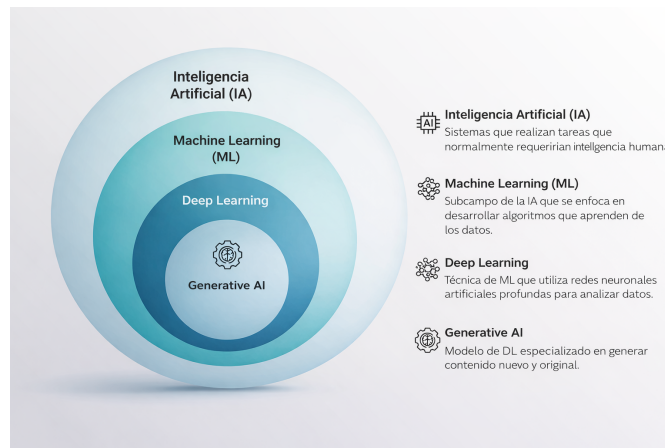


Figura 2.1: Relación jerárquica entre Inteligencia Artificial, Machine Learning, Deep Learning e IA Generativa.

2.1.1 Machine Learning

Machine learning (ML) puede definirse como un subcampo de la inteligencia artificial orientado al desarrollo de métodos computacionales que permiten a los sistemas aprender patrones a partir de datos y mejorar su desempeño en una tarea específica sin ser explícitamente programados para cada regla o escenario. En

lugar de basarse en instrucciones determinísticas predefinidas, estos modelos construyen representaciones matemáticas a partir de ejemplos, identificando regularidades estadísticas que luego se utilizan para realizar predicciones, clasificaciones o inferencias sobre nuevos datos [3].

Desde una perspectiva académica, el aprendizaje automático se fundamenta en principios de la estadística, la optimización y la teoría de la computación. Su objetivo central es generalizar a partir de la experiencia, minimizando el error de predicción mediante la estimación de funciones que modelen la relación entre las variables de entrada y de salida. Los enfoques de ML se clasifican en tres grandes categorías [3]:

- **Aprendizaje supervisado:** Los modelos se entrenan con datos etiquetados, es decir, pares de entrada y salida esperada. Se utilizan para tareas de clasificación (asignar categorías) y de regresión (predecir valores numéricos).
- **Aprendizaje no supervisado:** Los modelos trabajan con datos no etiquetados, buscando descubrir estructuras ocultas. Las aplicaciones típicas incluyen el clustering (agrupamiento), la detección de anomalías y la identificación de reglas de asociación.
- **Aprendizaje por refuerzo:** Los modelos aprenden mediante la interacción con un entorno, recibiendo señales de recompensa o penalización que guían su comportamiento hacia una estrategia óptima.

2.2 Minería de Datos Educativos

La Minería de Datos Educativos (EDM, *Educational Data Mining*) es un campo de investigación que se ocupa del desarrollo de métodos para explorar los datos provenientes de entornos educativos y utilizarlos para comprender mejor a los estudiantes y los contextos en los que aprenden. Formalmente, se define como la aplicación de técnicas de minería de datos a procesos educativos, con el fin de extraer conocimiento útil a partir de grandes volúmenes de información [1]. En una encuesta amplia del campo, Romero y Ventura presentan una visión actualizada de las principales técnicas, herramientas y aplicaciones de la EDM, así como de su relación con el área emergente de Learning Analytics [9].

Como se ilustra en la Figura 2.2, el proceso de EDM sigue una serie de pasos que van desde la recolección de datos hasta la obtención de conocimiento útil para la toma de decisiones.

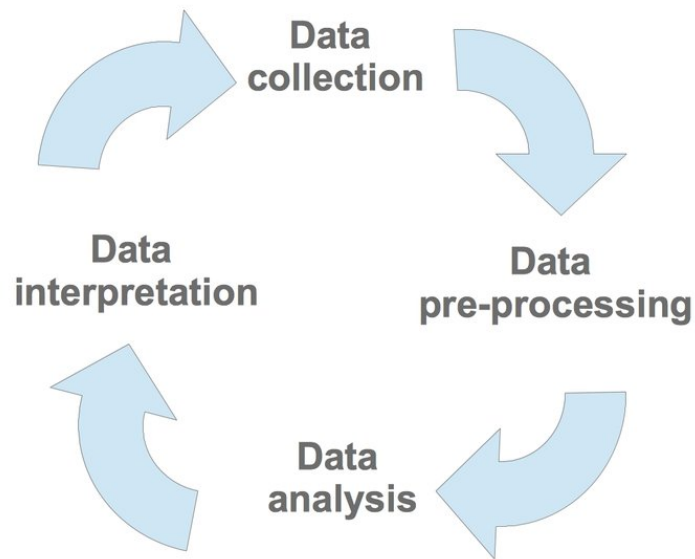


Figura 2.2: Pasos principales del proceso de minería de datos educativos [10].

Más allá de las técnicas específicas que emplea, lo que distingue a la EDM es el tipo de problemas que busca resolver en el contexto educativo. Entre las aplicaciones más relevantes se encuentran [9]:

- **Predicción del rendimiento académico:** Estimar el desempeño futuro de un estudiante a partir de su historial, lo que permite intervenciones tempranas en casos de riesgo.
- **Detección de deserción:** Identificar patrones que indican una alta probabilidad de abandono, para que las instituciones puedan tomar medidas preventivas.
- **Sistemas de recomendación:** Sugerir materias, recursos o estrategias de estudio personalizadas según el perfil y la trayectoria del estudiante.
- **Análisis de trayectorias:** Comprender los caminos que recorren los estudiantes a través del currículo, identificando cuellos de botella y patrones de éxito.
- **Mejora del diseño curricular:** Evaluar la efectividad de planes de estudio y detectar oportunidades de optimización basándose en datos reales.

Esta última aplicación —los sistemas de recomendación y el análisis de trayectorias— es particularmente relevante para el presente proyecto, ya que constituye la base sobre la que se construye la interfaz conversacional propuesta.

2.2.1 Análisis Curricular

Una aplicación específica de la EDM es el **análisis curricular**, que se enfoca en recopilar, analizar y visualizar datos a nivel de programa y curso para apoyar la toma de decisiones y la mejora de la calidad educativa. Este enfoque es particularmente valioso para las instituciones de educación superior, donde la complejidad de los planes de estudio, las correlaciones entre asignaturas y las trayectorias individuales de los estudiantes generan vastos volúmenes de datos que, cuando se procesan adecuadamente, revelan patrones y oportunidades de mejora.

El análisis curricular permite a las instituciones comprender en profundidad las **trayectorias estudiantiles**: qué caminos toman los estudiantes a través del currículo, cuáles son los cuellos de botella académicos, dónde se presentan las mayores tasas de reprobación y cómo diferentes decisiones de inscripción impactan en el desempeño general y en el tiempo de graduación. Esta comprensión es fundamental para ofrecer una orientación académica personalizada y efectiva.

Una técnica particularmente relevante en este contexto es el **Process Mining**, que permite descubrir, modelar y analizar procesos a partir de registros de eventos [11]. En una revisión sistemática reciente, Calegari y Delgado analizan la aplicación del Process Mining al análisis curricular —lo que denominan *curriculum mining*— identificando cinco objetivos principales de investigación, entre ellos el descubrimiento de trayectorias educativas y la detección de desviaciones en los recorridos de los estudiantes respecto a los planes teóricos [12]. Como se ilustra en la Figura 2.3, esta disciplina combina técnicas de minería de procesos con datos educativos para ofrecer una visión integral de cómo los estudiantes realmente avanzan a lo largo del currículo.

Entre los hallazgos de esta revisión se destacan desafíos como la necesidad de estandarización para facilitar el análisis entre distintas universidades, una mejor integración de técnicas de minería de procesos y de datos, y herramientas más accesibles para los actores educativos [12].

En el contexto del presente proyecto, el análisis curricular es un concepto clave para comprender cómo los estudiantes planifican y desarrollan su trayectoria académica. La interfaz conversacional propuesta utiliza los datos y patrones identi-

Graphical Abstract



Figura 2.3: Visión general de la aplicación de Process Mining al análisis curricular. Adaptado de Calegari y Delgado [12].

ficados mediante el análisis curricular para ofrecer recomendaciones personalizadas, asegurando que cada sugerencia respete las restricciones curriculares (correlativas, créditos, etc.) y se alinee con las trayectorias más exitosas observadas en la institución.

2.2.2 Proyecto de referencia: IA Aplicada a la Planificación de Estudios

Como ejemplo concreto de la aplicación de EDM al análisis curricular, cabe destacar el proyecto de grado “IA Aplicada a la Planificación de Estudios” desarrollado en la Universidad ORT Uruguay [13]. Este proyecto busca, mediante técnicas de IA y EDM, desarrollar un planificador académico capaz de ofrecer recomendaciones personalizadas a los estudiantes sobre qué asignaturas cursar cada semestre, considerando el rendimiento previo, las restricciones curriculares y las características individuales del estudiante.

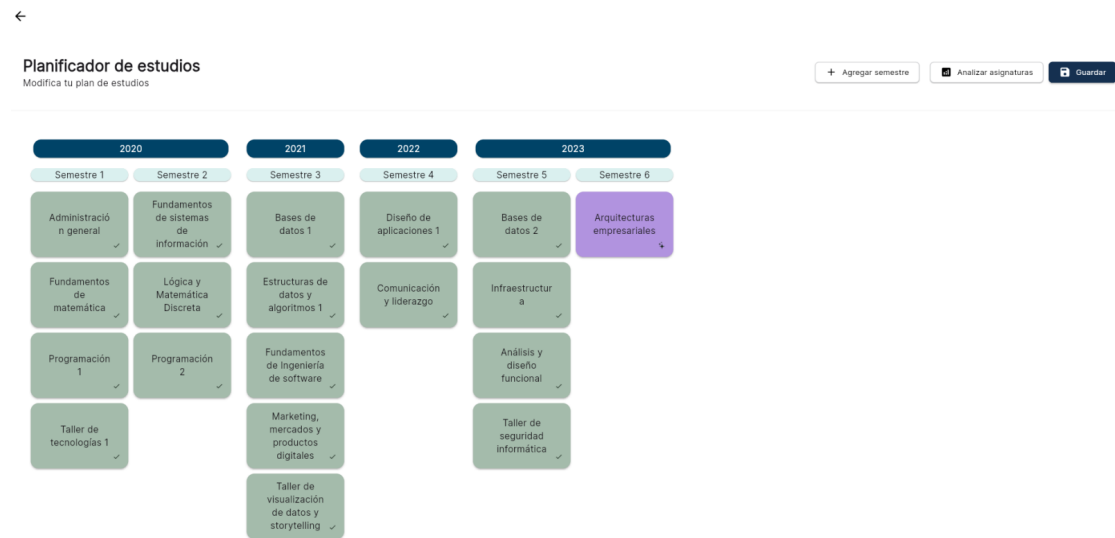


Figura 2.4: Interfaz del planificador de estudios [13].

Para la generación de recomendaciones, el proyecto implementó tres algoritmos complementarios de ML:

- **Random Forest:** Un método basado en ensambles que construye múltiples árboles de decisión y combina sus predicciones [14]. Se utilizó para predecir el rendimiento esperado del estudiante a partir de su historial académico.
- **Process Mining:** Se aplicó para modelar el flujo real de los estudiantes a lo largo del currículo, identificando trayectorias de éxito y desviaciones respecto a los planes teóricos [11].
- **Sequential Pattern Mining:** Se utilizó para descubrir secuencias frecuentes del orden en que los estudiantes cursan las asignaturas, identificando las trayectorias más comunes y exitosas.

Este proyecto constituye un antecedente directo del presente trabajo. Ambos buscan resolver el problema de la planificación académica mediante EDM, pero difieren en el enfoque tecnológico: mientras el proyecto anterior utiliza modelos tradicionales de ML desplegados en AWS, el presente trabajo opta por una arquitectura RAG con LLMs ejecutados localmente, lo que aporta una perspectiva complementaria basada en el lenguaje natural.

2.3 Interfaces Conversacionales

Las interfaces conversacionales son sistemas diseñados para permitir la interacción entre humanos y máquinas a través del lenguaje natural, ya sea oral o escrito. Su evolución ha sido notable, pasando de los primeros chatbots basados en reglas predefinidas y con capacidades limitadas a los sofisticados asistentes virtuales actuales que aprovechan los avances en inteligencia artificial. El propósito fundamental de estas herramientas es superar las barreras de las interfaces tradicionales, creando una experiencia de usuario más intuitiva y accesible.

Formalmente, las interfaces conversacionales son tecnologías que emplean inteligencia artificial, procesamiento de lenguaje natural y aprendizaje automático para desarrollar sistemas que simulan conversaciones humanas, permitiendo que los computadores comprendan y respondan a las entradas de los usuarios de una manera más natural e intuitiva [2]. El objetivo de estas herramientas, como los chatbots o asistentes virtuales, es facilitar el acceso a la información y el soporte, reduciendo la fricción en la interacción humano-máquina.

Los avances recientes en inteligencia artificial generativa, y en particular en los modelos de lenguaje de gran tamaño (LLMs) que se describen en la siguiente subsección, han potenciado significativamente estas interfaces, permitiéndoles generar desde respuestas simples hasta conversaciones complejas con coherencia y contextualización. Un estudio reciente encontró que una gran mayoría de usuarios (70 %) prefiere las interfaces conversacionales a los métodos tradicionales (como el correo electrónico o el soporte telefónico), citando factores como la **conveniencia, la eficiencia y la personalización** como razones clave para su elección [2]. Esta alta preferencia subraya el potencial de las interfaces conversacionales para transformar la interacción entre el usuario y la tecnología.

En el contexto educativo, esto es particularmente relevante: una interfaz conversacional puede ofrecer **asistencia personalizada disponible 24/7**, lo que mejora significativamente la experiencia del estudiante. Permite responder consultas recurrentes sin sobrecargar al personal administrativo y proporciona un canal directo de orientación académica. Estos beneficios la hacen especialmente valiosa para tareas como la planificación de estudios, donde los estudiantes requieren orientación personalizada basada en su situación académica particular.

En la Figura 2.5 se presenta un esquema general del funcionamiento de una interfaz conversacional moderna. El usuario formula una consulta en lenguaje natural a través de la interfaz de chat. Esta consulta es procesada por un modelo de lenguaje (LLM), que puede complementarse con una base de conocimiento exter-

na mediante una arquitectura RAG para fundamentar su respuesta en información verificable. Finalmente, el sistema genera una respuesta en lenguaje natural que se presenta al usuario.

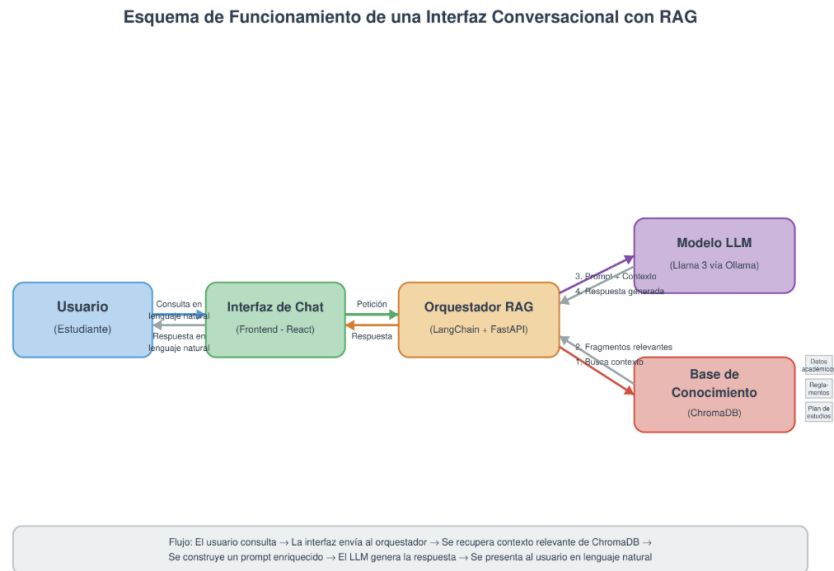


Figura 2.5: Esquema general del funcionamiento de una interfaz conversacional con arquitectura RAG.

2.3.1 Large Language Models (LLMs)

Los modelos de lenguaje de gran tamaño (Large Language Models, o LLMs) son la base de la tecnología de las interfaces conversacionales modernas. Son redes neuronales profundas, basadas en la arquitectura **Transformer**, que son entrenadas con enormes cantidades de texto para comprender y generar lenguaje humano con una fluidez sin precedentes [4, 15]. El desarrollo de un LLM competente generalmente se divide en dos fases críticas: el preentrenamiento y el postentrenamiento o ajuste fino.

2.3.1.1 Pre-entrenamiento

La fase inicial consiste en el preentrenamiento, en el que el modelo aprende patrones, gramática, razonamiento básico y conocimiento general a partir de un

corpus masivo de datos de texto no etiquetados. El objetivo de entrenamiento más común es la **predicción del siguiente token** (next-token prediction), en la que el modelo debe predecir la siguiente palabra de una secuencia, dado el contexto previo. Este proceso, aunque simple en su formulación, obliga al modelo a desarrollar una comprensión implícita de la estructura del lenguaje y del mundo para realizar predicciones precisas de manera consistente [15].

2.3.1.2 Post-entrenamiento y Ajuste Fino (Fine-Tuning)

Una vez preentrenado, un modelo base se somete a una etapa de “alineación” o ajuste fino para especializarlo en tareas concretas, como seguir instrucciones o mantener un diálogo coherente. Este post-entrenamiento es crucial para dar forma al comportamiento del modelo [16]. Existen varias técnicas para lograrlo:

- **Ajuste Fino Supervisado (Supervised Fine-Tuning - SFT):** Es un método ampliamente adoptado en el que el modelo se entrena con un conjunto de datos de alta calidad que contiene ejemplos de las respuestas deseadas. A menudo, se utiliza un proceso de “destilación”, en el que el modelo aprende a imitar los razonamientos o respuestas generadas por un modelo más avanzado y capaz [17, 18].
- **Aprendizaje por Refuerzo (Reinforcement Learning - RL):** Este paradigma permite que el modelo aprenda directamente a partir de señales de recompensa, que pueden ser verificables (por ejemplo, la corrección de un problema matemático) o basarse en preferencias humanas (un enfoque conocido como RLHF - Reinforcement Learning from Human Feedback) [17, 16]. Algoritmos como la Optimización de Políticas Proximales (PPO) y, más recientemente, la Optimización de Políticas Relativas a Grupos (GRPO) y la Optimización Directa de Preferencias (DPO) se han vuelto populares para alinear LLMs, ya sea en configuraciones offline, semi-online u online [16].
- **Ajuste Fino Eficiente en Parámetros (Parameter-Efficient Fine-Tuning - PEFT):** En contraste con el ajuste de todos los parámetros del modelo (full-parameter training), las técnicas PEFT, como LoRA (Low-Rank Adaptation), proponen una alternativa mucho más económica. LoRA modifica el comportamiento del modelo entrenando únicamente una pequeña cantidad de nuevos parámetros, que se añaden a la arquitectura original [17]. Este enfoque no solo reduce drásticamente los costos computacionales, sino que también ha demostrado ser sorprendentemente eficaz. Investigaciones recientes sugieren que LoRA es especialmente hábil para adaptar el modelo

a la **estructura y el formato** del razonamiento deseado, preservando en gran medida el conocimiento fundamental del modelo base y logrando un rendimiento competitivo con el de un entrenamiento completo [17, 18]. La Figura 2.6 ilustra el proceso general de fine-tuning para LLMs.

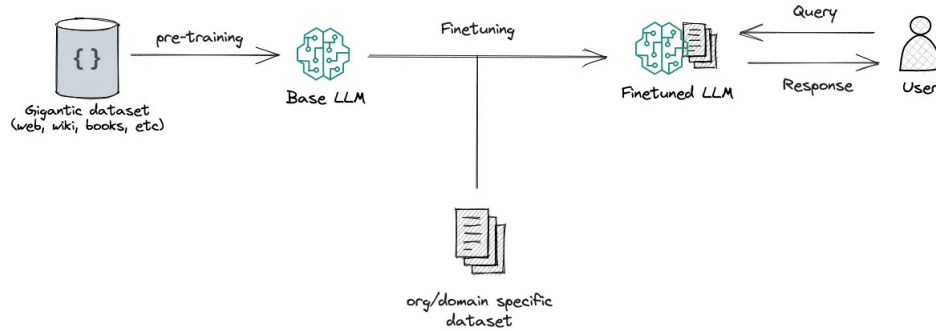


Figura 2.6: Ilustración del proceso de fine-tuning para LLMs [19].

La aplicación de estas técnicas sobre modelos base como los de las familias Llama, Qwen o DeepSeek es lo que ha dado lugar a los asistentes conversacionales y modelos de razonamiento avanzados que se utilizan hoy en día.

2.3.1.3 Ollama: Ejecución Local de Modelos

Aunque los LLMs más conocidos (como GPT-4 o Claude) se ejecutan en servidores remotos mediante APIs de pago, existe una alternativa importante: ejecutar LLMs abiertos localmente en máquinas convencionales. Para este propósito, la herramienta **Ollama** proporciona un entorno simplificado para la gestión de la descarga, la instalación y la ejecución de LLMs de código abierto.

Ollama es una aplicación de escritorio que abstrae la complejidad de configurar marcos de inferencia como ONNX Runtime o llama.cpp. Su principal ventaja es permitir experimentar con modelos sin costos por consulta, manteniendo los datos locales (lo cual es importante para la privacidad) y sin depender de la conexión a internet para la inferencia.

Entre los modelos disponibles en Ollama, la familia **Llama 3** de Meta destaca por su rendimiento competitivo frente a modelos propietarios. Llama 3 se basa en

una arquitectura de Transformer estándar densa, optimizada para la estabilidad y la escalabilidad. Sus variantes más compactas (8 billones de parámetros - 8B) ofrecen un equilibrio ideal entre capacidad cognitiva y eficiencia computacional, lo que permite su ejecución en hardware de consumidor.

Un factor crucial es que Llama 3 fue entrenado con un presupuesto de computación superior al óptimo, lo que se tradujo en un excelente rendimiento durante la inferencia [20]. Esto significa que sus versiones pequeñas no sacrifican significativamente la calidad por el tamaño. Además, Ollama ofrece tanto versiones base (preentrenadas) como versiones ajustadas por instrucciones, lo que proporciona flexibilidad para distintos casos de uso.

Esta arquitectura permite que investigadores y desarrolladores realicen experimentos reproducibles sin estar sujetos a las limitaciones de las APIs comerciales, lo cual resulta especialmente valioso en contextos académicos donde la reproducibilidad y el acceso abierto son fundamentales.

2.3.1.4 Modelo: DeepSeek

DeepSeek (DS) es un modelo de lenguaje grande (LLM) reciente que ha captado una atención significativa dentro y fuera de la comunidad de IA desde el lanzamiento de su última versión el 20 de enero de 2025 [21]. Lo que distingue a DeepSeek es su capacidad para lograr, en algunas tareas de referencia, resultados iguales o mejores que los de los principales modelos de la industria (como ChatGPT de OpenAI), pero con solo una fracción del costo de entrenamiento [21].

Por ejemplo, se ha reportado que DeepSeek-R1 muestra una precisión superior en problemas matemáticos complejos en comparación con ChatGPT y Gemini [21]. Además, investigadores han señalado su capacidad para resolver problemas de codificación complejos que otros modelos no han logrado resolver [21].

En el estudio comparativo de Gao et al., se evaluó DeepSeek-R1 frente a otros cuatro LLM populares: Claude-3.5-sonnet, Gemini-1.5-flash, GPT-4o-mini y Llama-3.1-8b [21]. La comparación se centró en tareas de clasificación de texto, específicamente en la clasificación de autoría (determinando si un texto fue escrito por un humano o por una IA) y en la clasificación de citas académicas [21].

Los resultados generales de la comparación de precisión muestran que DeepSeek supera a Gemini, GPT y Llama en la mayoría de los casos, pero se sitúa por debajo de Claude [21]. En términos de los rankings promedio de las cuatro tareas experimentales, Claude obtuvo el primer lugar (1.25), seguido por DeepSeek (2.25)

[21]. El estudio también destaca otras características: DeepSeek es comparativamente más lento que los demás modelos, pero tiene un bajo costo de uso, mientras que Claude, el de mejor rendimiento, es considerablemente más caro que las demás alternativas [21].

2.3.1.5 Modelo: Qwen

Qwen 2.5 es un LLM desarrollado por la división de investigación en inteligencia artificial de Alibaba [22].

La “personalidad” o el estilo de este modelo se deriva directamente de su entrenamiento y de sus objetivos de diseño. A diferencia de otros modelos, el corpus de entrenamiento de Qwen 2.5 fue masivo, con un tamaño estimado de 18 billones de tokens, y consistía principalmente en contenido técnico y relacionado con el código. El objetivo explícito de Alibaba con este enfoque era lograr la **“fiabilidad empresarial y salidas estructuradas”** [22].

Como resultado directo de esta filosofía de entrenamiento técnico, Qwen 2.5 desarrolló un estilo de escritura muy distintivo. El análisis de sus textos revela que produce consistentemente salidas estructuradas y formales. Esto se manifiesta en frases que tienden a ser más largas e intrincadas, con una dicción formal y precisa. En resumen, el tono de Qwen 2.5 es metódico y profesional. Esta huella lingüística es tan definida que, en estudios comparativos, los clasificadores de aprendizaje automático (como un SVM) lograron identificar los textos generados por Qwen 2.5 con una **tasa de éxito del 99 %** [22].

2.3.2 Generación Aumentada por Recuperación (RAG)

La Generación Aumentada por Recuperación (Retrieval-Augmented Generation o RAG) es una arquitectura que mejora la capacidad de los LLMs al permitirles acceder a información externa y actualizada en tiempo real. Se ha consolidado como un enfoque prominente que combina las fortalezas de los modelos de recuperación de información con los de generación de lenguaje [4].

En lugar de depender únicamente del conocimiento interno del modelo, que es estático y puede quedar obsoleto, la arquitectura RAG funciona de la siguiente manera: el modelo primero consulta una base de conocimiento externa (como

una colección de documentos o una base de datos) para recuperar fragmentos de información relevantes para la consulta del usuario. Posteriormente, esta información recuperada se utiliza como contexto adicional para que el modelo de lenguaje genere una respuesta más precisa, fundamentada y actualizada.

Este enfoque es particularmente útil para reducir las alucinaciones, basar las respuestas en conocimiento verificable y permitir la personalización de los resultados. El objetivo principal del componente de recuperación en un sistema RAG es servir de manera efectiva al modelo de lenguaje que consumirá los resultados para cumplir con la tarea asignada [4].

2.3.2.1 Base de datos de vectores

La arquitectura RAG depende fundamentalmente de un mecanismo eficiente para encontrar información relevante en una gran base de conocimiento. Esta búsqueda no se basa en palabras clave exactas, sino en la similitud semántica, lo cual se logra mediante el uso de *embeddings* (representaciones vectoriales de texto) y bases de datos especializadas en su almacenamiento y consulta: las bases de datos vectoriales.

Para este proyecto, se seleccionó **ChromaDB** como base de datos vectorial. ChromaDB es una base de datos de código abierto diseñada específicamente para aplicaciones de inteligencia artificial, optimizada para almacenar y consultar *embeddings* de manera eficiente. Su función principal dentro de la arquitectura RAG es indexar los vectores correspondientes a los fragmentos de documentos relevantes (como planes de estudio, reglamentos, etc.) y permitir búsquedas de similitud rápidas al realizar consultas.

La elección de ChromaDB se justifica por varias razones clave:

- **Facilidad de uso e integración:** ChromaDB ofrece una API sencilla en Python y se integra nativamente con frameworks como LangChain, lo que simplifica considerablemente el proceso de desarrollo e implementación del flujo RAG.
- **Optimización para RAG:** Está diseñada para los flujos de trabajo de RAG, facilitando el almacenamiento no solo de los vectores, sino también de los metadatos y del texto original asociado, lo cual es crucial para construir el contexto que se envía al LLM.

- **Despliegue local:** Permite una ejecución local (in-memory o en disco) sin necesidad de configurar servidores complejos, lo cual es ideal para una prueba de concepto y se alinea con la decisión de usar Ollama para la ejecución local del LLM.
- **Código abierto:** Su naturaleza open-source elimina costos de licencia y fomenta la transparencia y la colaboración comunitaria.

El proceso de almacenamiento y recuperación de información con ChromaDB sigue los pasos estándar de un modelo RAG: primero, los documentos se dividen en fragmentos (chunks), luego se generan los embeddings para cada fragmento usando un modelo adecuado, y finalmente, estos embeddings junto con el texto original y metadatos se almacenan en ChromaDB. Durante la consulta, el embedding de la pregunta del usuario se utiliza para buscar los embeddings más cercanos (semánticamente similares) en la base de datos, recuperando así los fragmentos de texto más relevantes para responder a la pregunta.

Se exploraron otras bases de datos vectoriales como **Pinecone** y **Weaviate**. Pinecone, aunque muy potente, es principalmente un servicio gestionado en la nube con costos significativos por consulta. Weaviate, si bien también es de código abierto y muy capaz, presentaba una curva de aprendizaje y una complejidad de configuración inicial mayores que las ofrecidas por ChromaDB para el alcance de este proyecto. Dado el enfoque en una prueba de concepto eficiente, ChromaDB emergió como la opción más pragmática, combinando facilidad de uso con funcionalidades adecuadas para RAG.

2.3.2.2 LangChain: Orquestación del flujo RAG

Para implementar la lógica central de la arquitectura RAG y gestionar la interacción entre sus componentes, se seleccionó el framework **LangChain**. LangChain es un marco de desarrollo de código abierto diseñado específicamente para simplificar la creación de aplicaciones impulsadas por LLMs, con enfoque en la composabilidad y la gestión del contexto.

La implementación de un sistema RAG implica coordinar varios pasos: procesar la entrada del usuario, recuperar información relevante de una base de conocimiento, construir un prompt enriquecido con esa información, interactuar con el LLM y, finalmente, presentar la respuesta. LangChain facilita enormemente este proceso al proporcionar componentes modulares y abstracciones para cada una de estas etapas.

Las razones principales para elegir LangChain fueron:

- **Abstracción de componentes:** Ofrece interfaces estandarizadas para interactuar con diferentes LLMs (a través de Ollama en nuestro caso), bases de datos vectoriales (como ChromaDB) y herramientas de procesamiento de texto. Esto desacopla la lógica de la aplicación de las implementaciones específicas.
- **Construcción de cadenas (Chains):** El concepto central de “cadenas” en LangChain permite definir secuencias de operaciones (por ejemplo, recuperar documentos y luego pasarlos al LLM) de manera declarativa y sencilla. Esto simplifica la implementación del flujo RAG completo.
- **Gestión de prompts:** Proporciona herramientas para crear, gestionar y optimizar plantillas de prompts, lo cual es crucial para guiar eficazmente al LLM y asegurar que utilice el contexto recuperado de manera adecuada.
- **Integración con el ecosistema:** LangChain se integra fluidamente con el ecosistema de Python para IA, incluyendo librerías como FastAPI y bases de datos como ChromaDB, lo que permite una implementación coherente con el resto del stack tecnológico seleccionado.

En esencia, LangChain actúa como el “pegamento” que une los distintos componentes de la arquitectura RAG, permitiendo orquestar el flujo de información desde la consulta del usuario hasta la generación de la respuesta final por parte del LLM, enriquecida con el contexto recuperado de ChromaDB.

2.3.2.3 LlamaIndex

Como alternativa a LangChain, **LlamaIndex** es un marco de trabajo centrado en datos, diseñado para simplificar la ingesta, la estructuración y el acceso a datos privados o específicos de un dominio para su uso con LLMs [23]. Su propósito principal es actuar como una interfaz intermedia entre las fuentes de conocimiento (documentos, bases de datos, APIs) y el modelo de lenguaje, proporcionando herramientas para la ingesta de datos desde diversas fuentes, la indexación mediante vectores u otros tipos de índices (listas, árboles, grafos de conocimiento), y la consulta de alto nivel sobre los datos indexados [23].

LlamaIndex está particularmente optimizado para casos de uso RAG, donde la calidad y la relevancia de la información recuperada son cruciales para mejorar la

precisión y reducir las alucinaciones del modelo de lenguaje [23]. Para el presente proyecto se optó por LangChain debido a su mayor flexibilidad en la construcción de cadenas de procesamiento y a su integración más directa con Ollama, aunque LlamaIndex representa una alternativa válida, especialmente para proyectos con necesidades complejas de indexación de datos.

2.3.2.4 Proyectos de referencia con arquitectura RAG

En el ámbito universitario, se han desarrollado proyectos que implementan la arquitectura RAG para facilitar el acceso a la información institucional mediante interfaces conversacionales. A continuación se presentan dos antecedentes relevantes realizados en la Universidad ORT Uruguay que validan el uso de esta arquitectura.

Proyecto AURA (Asistente Universitario de Recursos y Aprendizaje)

El proyecto AURA buscó mejorar el acceso a la información institucional y la comunicación en el entorno universitario, mediante un asistente virtual inteligente que centraliza información sobre carreras, becas, trámites y servicios académicos (Figura 2.7). El sistema también proporciona herramientas pedagógicas (como flashcards, ensayos y mapas conceptuales) y funcionalidades de seguimiento.

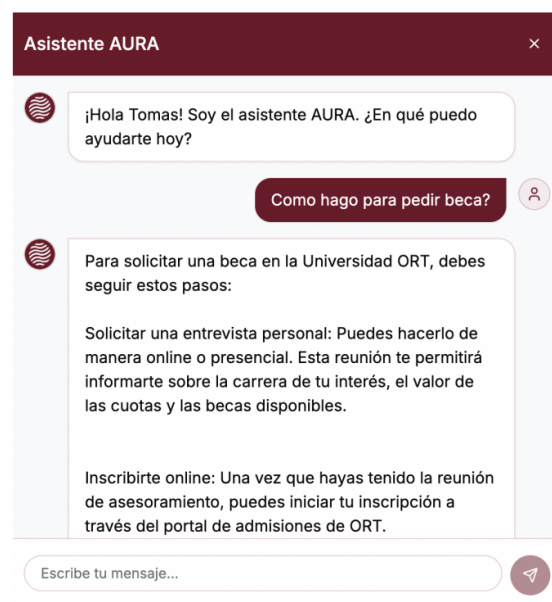


Figura 2.7: Interfaz chat Proyecto AURA [24].

AURA se construyó con una arquitectura modular basada en microservicios: backend en Python con FastAPI, frontend en React (Next.js), múltiples bases de datos especializadas (PostgreSQL, Cassandra, ChromaDB, Redis) y modelos GPT de OpenAI combinados con LangChain y Ollama sobre una arquitectura Agentic RAG. El despliegue se gestionó con Docker y GitHub Actions [24].

Relevancia para el proyecto actual: AURA proporciona un caso de estudio directo sobre la implementación de la arquitectura RAG en un contexto universitario. El presente proyecto comparte tecnologías clave (FastAPI, ChromaDB, LangChain), pero se enfoca específicamente en la planificación académica individual, con un alcance más acotado y con énfasis en modelos open source locales.

Proyecto Chat Your Data

Este proyecto desarrolló una plataforma conversacional basada en IA que permite a las organizaciones interactuar de forma natural con sus datos internos, facilitando la búsqueda, el análisis y la generación automatizada de reportes mediante consultas en lenguaje natural.

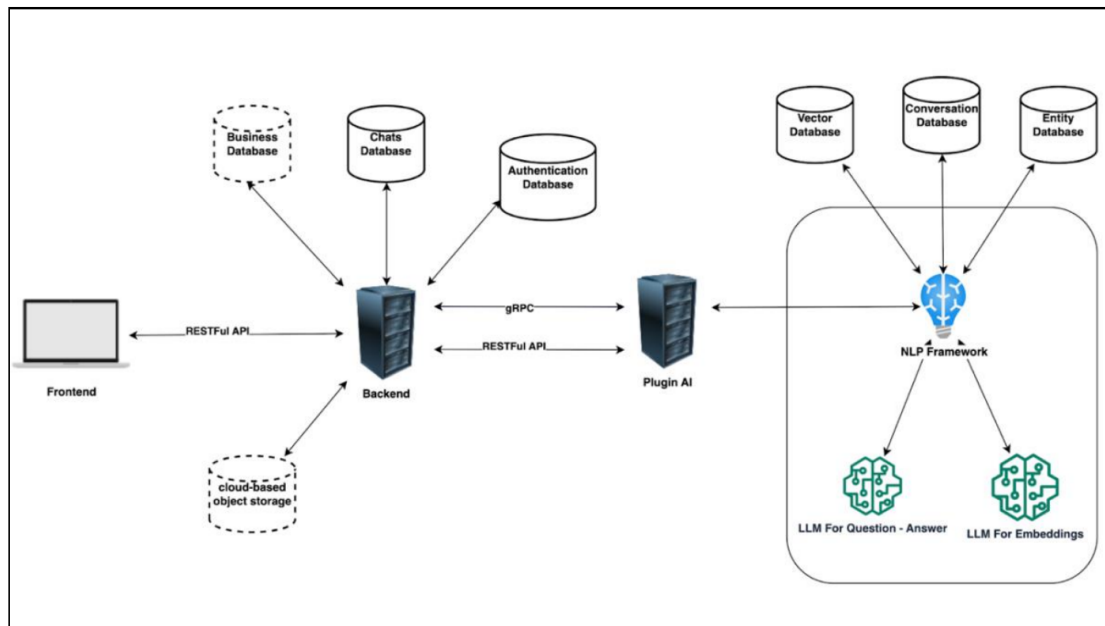


Figura 2.8: Arquitectura a alto nivel de Chat Your Data [25].

Como se ilustra en la Figura 2.8, la solución se implementó bajo una arquitectu-

ra modular contenerizada: frontend en React, backend en Nest.js y un componente de IA denominado PluginAI. Se emplearon modelos GPT (OpenAI) combinados con LangChain, y el despliegue se realizó en AWS con Docker [25].

Relevancia para el proyecto actual: Chat Your Data valida el enfoque RAG para realizar consultas en lenguaje natural sobre bases de datos internas. El presente proyecto adopta un paradigma similar, pero se centra en un dominio específico (la planificación académica) y utiliza infraestructura local en lugar de la nube.

2.3.3 De RAG a Agentes Inteligentes

Si bien la arquitectura RAG constituye la base conceptual del presente proyecto, la complejidad de las consultas académicas llevó a evolucionar hacia un enfoque más flexible. En lugar de un flujo RAG tradicional —donde la recuperación de información se inyecta directamente en el prompt del LLM—, el sistema adoptó una arquitectura basada en agentes inteligentes que emplean el patrón **ReAct** (Reasoning + Acting).

Este enfoque permite que el modelo no solo recupere información, sino que también razone sobre qué herramientas invocar en cada situación: buscar documentos, consultar la escolaridad del estudiante, validar prerrequisitos, entre otras. De esta manera, el agente asume las responsabilidades del RAG de forma controlada, ofreciendo respuestas más precisas y deterministas ante consultas que requieren múltiples fuentes de información. La implementación concreta de esta arquitectura se describe en detalle en el Capítulo 4.

2.4 Frameworks de desarrollo

En esta sección se presentan las tecnologías complementarias utilizadas para la construcción del backend y el frontend de la solución. A diferencia de las tecnologías centrales al problema (LLMs, RAG y sus componentes, descritas en las secciones anteriores), los frameworks aquí presentados constituyen la infraestructura de soporte sobre la cual se despliega la aplicación.

2.4.1 FastAPI

Para la construcción del backend de la solución, se decidió utilizar el lenguaje de programación **Python** y el framework **FastAPI**. Esta elección responde a una alineación estratégica con los objetivos del proyecto, especialmente en lo que respecta a la integración de algoritmos de inteligencia artificial y la eficiencia en el desarrollo.

La elección de **Python** se fundamenta en su robusto y extenso ecosistema de librerías para inteligencia artificial y ciencia de datos. Herramientas como LangChain, PyTorch y TensorFlow están optimizadas para este lenguaje, lo que simplifica enormemente la implementación de arquitecturas complejas como RAG. La legibilidad y versatilidad de Python también agilizan el ciclo de desarrollo, permitiendo una rápida transición desde la conceptualización teórica hasta la implementación práctica del código.

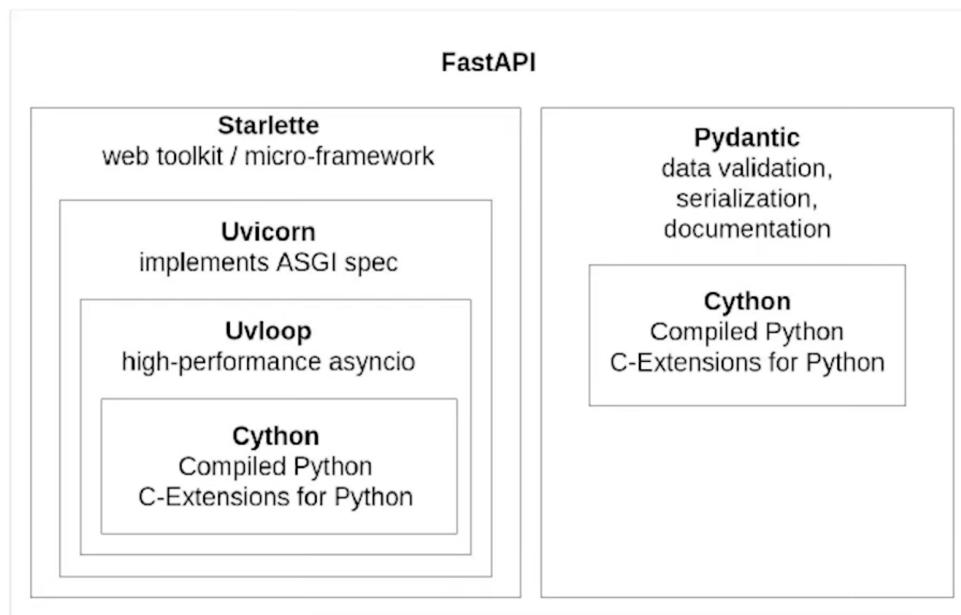


Figura 2.9: Arquitectura de FastAPI [26]

Dentro del ecosistema de Python, FastAPI (Figura 2.9) se destaca por su alto rendimiento, comparable al de frameworks en lenguajes compilados, y por sus capacidades nativas de programación asíncrona, lo cual es ideal para manejar operaciones de entrada/salida intensivas, como las consultas a LLMs. Además, su

arquitectura moderna y su estricta adherencia a los estándares RESTful facilitan la creación de endpoints bien definidos y una clara separación entre la lógica del algoritmo y el framework del backend.

2.4.2 Uvicorn

Uvicorn es la implementación de servidor ASGI (Asynchronous Server Gateway Interface) recomendada para FastAPI. Se describe como un servidor liviano y rápido, que basa su alto rendimiento en el uso de `uvloop` y `httptools` [26].

La especificación ASGI se desarrolló para cubrir el vacío que tenía Python en una interfaz estándar de bajo nivel para frameworks asíncronos (`asyncio`). Su objetivo es permitir que los frameworks web de Python compitan en rendimiento con plataformas como Node.js y Go, especialmente en contextos con alta carga de E/S [26]. A diferencia del estándar WSGI anterior, ASGI puede manejar protocolos modernos como WebSockets [26].

2.4.3 Node.js

Node.js es un entorno de ejecución de JavaScript del lado del servidor construido sobre el motor V8 de Chrome, diseñado específicamente para crear aplicaciones de red escalables [27]. Su característica fundamental es su modelo de **entrada/-salida (I/O) asíncrono y sin bloqueo**, gestionado por un **bucle de eventos (event loop)** [27].

A diferencia de los modelos tradicionales basados en hilos, Node.js utiliza un único hilo principal que delega las operaciones de I/O al sistema operativo, registrando callbacks para su ejecución posterior. Este enfoque permite manejar miles de conexiones simultáneas con un bajo consumo de memoria y recursos [27].

Node.js (Figura 2.10) se ha consolidado como una tecnología popular para el desarrollo de backend, especialmente para aplicaciones en tiempo real y APIs RESTful. Su ecosistema NPM facilita la incorporación de funcionalidades mediante módulos de código abierto, y permite utilizar JavaScript tanto en el frontend como en el backend, simplificando el stack tecnológico [27].

Node.js Architecture

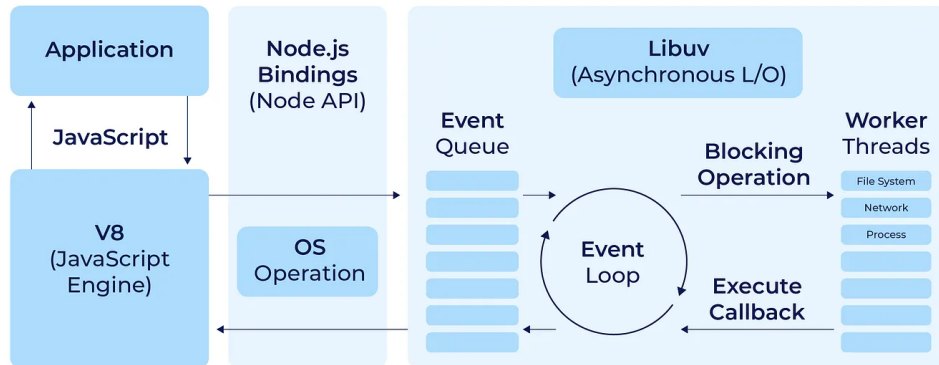


Figura 2.10: Arquitectura de Node.js [28]

2.4.4 Flask

Flask es un microframework para Python diseñado para ser ligero y fácil de extender [29]. Ofrece las herramientas básicas para gestionar rutas, solicitudes y respuestas en una aplicación web. Su filosofía minimalista permite iniciar proyectos rápidamente y resulta especialmente adecuada para aplicaciones pequeñas o medianas y APIs, donde se prefiere seleccionar e integrar las librerías manualmente [29].

2.4.5 Django

En contraste con los microframeworks, **Django** adopta un enfoque “baterías incluidas”, proveyendo un conjunto extenso de herramientas listas para usar [30]. Está diseñado para facilitar el desarrollo rápido de aplicaciones web complejas y seguras, siguiendo el patrón Modelo-Vista-Plantilla (MVT). Incluye un ORM integrado, un panel de administración automático, un sistema de rutas, un motor de plantillas y componentes de seguridad contra vulnerabilidades comunes [30]. Su enfoque lo hace particularmente adecuado para proyectos grandes que requieren una estructura sólida desde el inicio [30].

2.5 Plataformas de despliegue

2.5.1 Servidores

2.5.1.1 Servidores en la nube (Cloud)

Los servidores en la nube, según el NIST (Instituto Nacional de Estándares y Tecnología), se entienden como un modelo que permite el acceso a una red ubicua, de bajo costo y bajo uso. Esta red comparte un conjunto de recursos informáticos configurables que pueden aprovisionarse y liberarse rápidamente, con un mínimo esfuerzo de gestión o de interacción con el proveedor de servicios. [31]

Existen tres modelos de servicios en la nube: Infraestructura como Servicio (IaaS, Figura 2.11), Plataforma como Servicio (PaaS) y Software como Servicio (SaaS). A efectos de este proyecto, nos concentraremos en la infraestructura como servicio.

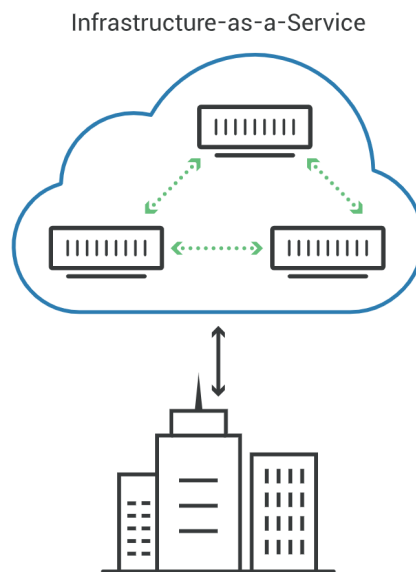


Figura 2.11: Infraestructura como servicio [32]

Amazon Web Services (AWS)

Amazon Web Services (AWS) es una plataforma de Amazon que ofrece servicios de IaaS (Figura 2.12). Se ofrecen múltiples servicios en diversas categorías, como

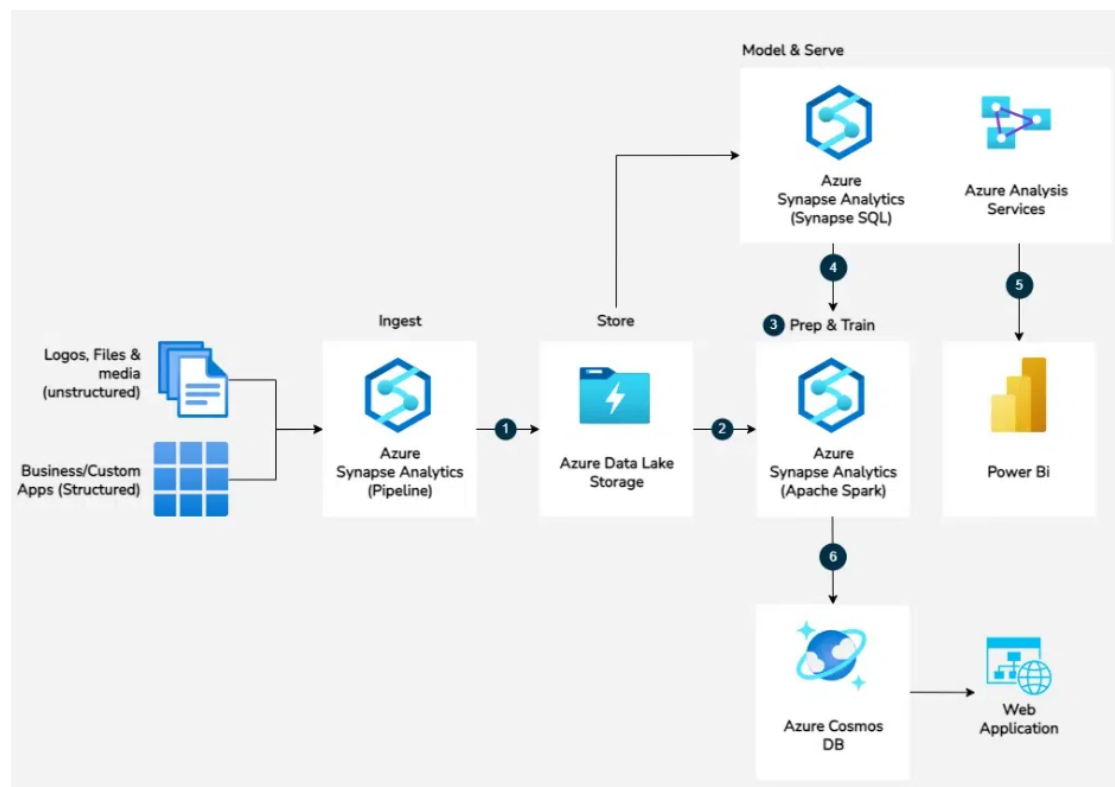


Figura 2.13: Arquitectura de Azure [38]

Google Cloud Platform (GCP) Services

Google inició la creación de una plataforma cloud de forma temprana, pero solo para uso interno. Esto hace que su plataforma sea menos conocida y utilizada (al segundo trimestre de 2022, tenía un 8 % del mercado, mientras que sus principales competidores, Amazon y Microsoft, tenían el 31 % y el 24 % respectivamente). [39]

La plataforma cuenta con más de 100 productos, entre ellos: Compute Engine (un IaaS de máquinas virtuales que se ejecutan en el centro de datos de Google), Cloud Storage (ofrece almacenamiento seguro, duradero y escalable para grandes conjuntos de datos sin estructura), Kubernetes Engien (un sistema de gestión y orquestación para ejecutar aplicaciones en contenedores Docker y clústeres de contenedores), Bigquery (un almacén de datos para agilizar las operaciones empresariales y extraer información), a su vez, ofrece servicios de computación sin servidor proporcionando herramientas y servicios para la ejecución de cargas de trabajo basadas en eventos, como Cloud Functions (para crear funciones que ma-

nejen eventos en la nube), Cloud Run (para administrar y ejecutar aplicaciones en contenedores y WorkFlows para orquestar API y productos sin servidor), entre otro sin fin de funcionalidades.

Comparativa

Entendemos que es pertinente realizar una comparativa entre estas tres plataformas de computación en la nube. Las tres soluciones representan la mayoría del mercado actual de servicios cloud, tanto en infraestructura (IaaS) como en plataforma (PaaS), y son frecuentemente evaluadas en investigaciones académicas por su impacto en la adopción tecnológica en organizaciones de distintas escalas.

AWS se posiciona como la plataforma más madura y completa, destacándose por su amplia gama de servicios y su presencia global en múltiples regiones. Su enfoque flexible en modelos de facturación —on-demand, reserved y spot instances— le permite adaptarse a diversos escenarios empresariales, lo que la convierte en la opción predilecta para empresas que priorizan la estabilidad y la diversidad de herramientas. Microsoft Azure, en cambio, presenta ventajas en entornos corporativos con infraestructura basada en productos de Microsoft. La integración nativa con servicios como Active Directory y Visual Studio, junto con políticas de licenciamiento híbrido, hace de Azure una alternativa particularmente atractiva para organizaciones que buscan continuidad entre sistemas locales y la nube.[40]

Por su parte, Google Cloud Platform se destaca por su enfoque innovador en inteligencia artificial y machine learning, así como por su infraestructura eficiente en términos de rendimiento y de migración en vivo de máquinas virtuales. No obstante, si bien GCP ha logrado avances significativos en escalabilidad y costos competitivos, su adopción en el segmento empresarial aún es menor que la de AWS y Azure. En términos de seguridad y disponibilidad, los estudios revisados concluyen que tanto AWS como Azure ofrecen garantías equivalentes, con estándares de cifrado robustos (AES-256) y acuerdos de nivel de servicio (SLA) superiores al 99,9 %, mientras que GCP mantiene métricas similares con una propuesta más centrada en entornos de desarrollo y análisis de datos. [41]

En cuanto a estas tres plataformas, podemos concluir que: la literatura coincide en que AWS ofrece la mayor madurez y amplitud de servicios; Azure destaca por su integración corporativa y adaptabilidad híbrida; y GCP sobresale en innovación tecnológica y eficiencia en costos, constituyendo tres modelos de referencia complementarios dentro del ecosistema de la computación en la nube [40] [41].

2.5.1.2 Servidor local (On-Premise)

El modelo on-premise (Figura 2.14) consiste en instalar todos los componentes de una solución informática en la propia empresa, utilizando sus propios servidores e infraestructura. Uno de los mayores beneficios del modelo cloud frente a este enfoque es que, a diferencia de los servidores on-premise, donde hay que realizar una inversión significativa en infraestructura, con un pequeño costo variable es posible tener un servidor que, a su vez, es altamente escalable.

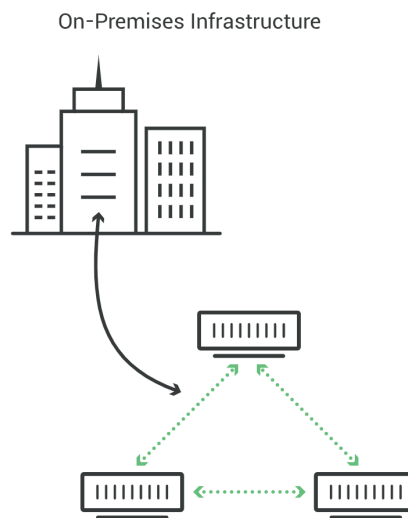


Figura 2.14: Infraestructura On-Premises [32]

2.5.1.3 Comparativa

El modelo de servidores locales o on-premise consiste en instalar todos los componentes de una solución informática en la propia empresa, utilizando sus propios servidores e infraestructura. En estos casos, las empresas son responsables de la seguridad, la disponibilidad y la gestión de dichos sistemas. [31] Las principales ventajas de tener servidores en la nube son la seguridad, la rapidez, el ahorro de costes en infraestructura, la flexibilidad, la eficiencia energética, la ausencia de gastos de mantenimiento, la fidelidad y la confiabilidad. A continuación, dejamos una tabla comparativa entre ambos modelos:

Factor Clave	IaaS	On-Premise
Ubicación del servidor	<i>Proveedor de Nube</i>	<i>Físico, Local</i>
Mantenimiento	Desde cualquier lugar	En el lugar
Apagones de servidor	Posible pero escalable	Posible
Demanda de recursos	Fácilmente escalable	Consume tiempo y esfuerzo
Instalación	Fácil y rápido	Consume mucho tiempo
Disponibilidad	Bajo demanda	Se debe comprar, instalar y desplegar
Inversión	Menos	Alto
Riesgos empresariales	Bajo	Alto
Memoria ilimitada	Sí	No
Respaldos	Oportuna	Requiere mayor esfuerzo por parte de la empresa
Capacidad para manejar cargas de trabajo	Alto	Bajo
Ahorro de energía	Viene incluido en el paquete que se haya contratado	Debe ser gestionado individualmente
Ahorro de costes	Alto	Bajo
Costo accesible para Pymes	Bajo	Alto
Mantenimiento de la infraestructura	Lo realiza el proveedor	Responsabilidad de la empresa
Mejora del ancho de banda	Bajo demanda	Exige nuevas implementaciones
Hardware	Responsabilidad del proveedor	Responsabilidad de la organización
Configuraciones	Estándar	Personalizable
Eliminación de malware	Fácil	Difícil
Vulnerabilidad a los ataques	Sí	Sí
Responsabilidad de mal funcionamiento	Proveedor de servicios	Organización

Tabla 2.1: Comparativa servidores on-Premise vs en la nube [42]

2.5.2 Empaquetado, aislamiento y ejecución de la aplicación

2.5.2.1 Docker

39

Docker es una plataforma de código abierto utilizada para construir, distribuir y ejecutar aplicaciones (Figura 2.15). A su vez, provee un servicio en la nube para

compartir aplicaciones. Uno de los elementos más utilizados de Docker son sus contenedores. A diferencia de las máquinas virtuales —que emulan un sistema operativo completo en el hardware del host, incluyendo su propio kernel, lo que implica un mayor consumo de recursos—, los contenedores comparten el kernel del sistema operativo anfitrión y empaquetan únicamente la aplicación junto con sus dependencias. Esto permite un inicio más rápido, un menor consumo de memoria y una mayor densidad de despliegue, lo que reduce significativamente los costos frente a las máquinas virtuales tradicionales [43].

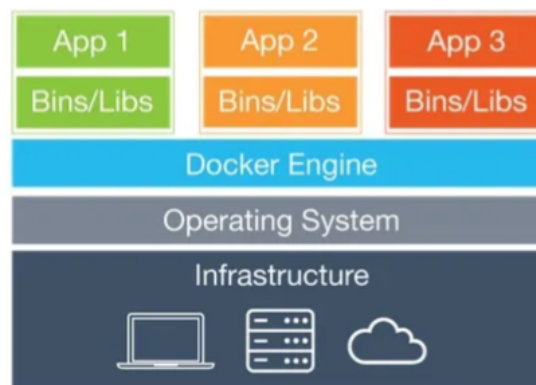


Figura 2.15: Arquitectura de los contenedores de Docker [43].

Las aplicaciones se empaquetan con todas sus dependencias en lo que se conoce como un contenedor. Los mismos operan de forma aislada por encima del kernel del sistema operativo.

2.5.2.2 Podman

Podman es una alternativa de código abierto a Docker que proporciona una plataforma para construir, ejecutar y gestionar contenedores. A diferencia de Docker, Podman no requiere un daemon en ejecución y puede ejecutar contenedores sin privilegios de root, lo que mejora la seguridad [35].

Las principales características de Podman incluyen: Compatibilidad con Docker: Podman es compatible con las imágenes y los contenedores de Docker, lo que permite una migración sencilla desde Docker. Ejecución sin root: Los contenedores pueden ejecutarse sin privilegios de administrador, lo que reduce la superficie de

ataque. Arquitectura sin daemon: No requiere un proceso daemon en ejecución, lo que simplifica la gestión y reduce el consumo de recursos. Compatibilidad con Docker Compose: A través de podman-compose, puede ejecutar configuraciones de Docker Compose.

Para este proyecto, se optó por Docker debido a su mayor madurez, su documentación extensa y su mejor integración con las herramientas de desarrollo empleadas. Sin embargo, Podman representa una alternativa viable para entornos en los que la seguridad y la ejecución sin privilegios son prioritarias.

2.6 Síntesis y Conexión con el Proyecto

El presente proyecto integra los conceptos del marco teórico presentado en una arquitectura concreta diseñada para mejorar la planificación de estudios en la Universidad ORT Uruguay. La solución articula los diferentes elementos teóricos de la siguiente manera:

1. **Minería de Datos Educativos:** El proyecto utiliza prácticas de EDM para recopilar, analizar y estructurar datos académicos históricos de la institución. Estos datos constituyen la base de conocimiento que alimenta el sistema de recomendaciones.
2. **Análisis Curricular:** Las trayectorias estudiantiles y las restricciones curriculares (correlativas, créditos, requisitos) se incorporan explícitamente en el sistema. Esto asegura que todas las recomendaciones respeten la estructura del plan de estudios y se alineen con patrones de éxito identificados en los estudiantes previos.
3. **Interfaces Conversacionales y LLMs:** Se implementa una interfaz conversacional potenciada por un LLM (Llama 3 vía Ollama) que permite a los estudiantes formular consultas en lenguaje natural sobre su planificación académica. El LLM interpreta las preguntas y genera respuestas comprensibles y personalizadas.
4. **Recuperación Aumentada por Generación (RAG):** La arquitectura RAG se emplea para fundamentar las respuestas del LLM en información verificable y actualizada. Los documentos institucionales (plan de estudios, reglamentos, normativas) se indexan en ChromaDB. Durante cada consulta, el sistema recupera los fragmentos más relevantes y los proporciona al LLM como contexto, lo que reduce las alucinaciones y garantiza respuestas confiables.

Esta síntesis conceptual se materializa en un stack tecnológico coherente (Python, FastAPI, LangChain, ChromaDB, Ollama + Llama 3) que permite desarrollar una prueba de concepto funcional, escalable y orientada a mejorar la experiencia del estudiante durante el proceso de planificación académica. La elección de ejecutar Llama 3 localmente a través de Ollama garantiza la reproducibilidad, la privacidad de los datos y la ausencia de costos operativos, lo cual se alinea con los valores académicos de transparencia y acceso abierto.

3 Planteamiento del problema

En este capítulo se describe el contexto del problema abordado, se identifican las necesidades que motivan el desarrollo de la solución y se definen los requisitos funcionales y no funcionales del sistema.

3.1 Contexto

Las instituciones de educación superior, como la Universidad ORT Uruguay, generan una gran cantidad de datos sobre el desempeño académico, la inscripción en cursos y las trayectorias de los estudiantes. Sin embargo, esta valiosa información a menudo no se aprovecha plenamente para ofrecer apoyo personalizado ni facilitar a los estudiantes la toma de decisiones académicas.

La planificación de estudios es un área crítica en la que esta carencia resulta evidente. Los estudiantes deben navegar por una compleja red de planes de estudio, correlaciones entre asignaturas y créditos, lo que genera un volumen considerable de datos. Actualmente, el acceso a la orientación personalizada depende de canales que pueden sobrecargarse con consultas recurrentes.

Como resultado, los estudiantes carecen de una herramienta accesible e intuitiva, disponible 24/7, que les proporcione orientación personalizada e instantánea sobre su carrera y les ayude a tomar decisiones informadas, como qué materias cursar el siguiente semestre.

La solución propuesta consiste en el diseño y desarrollo de una **prueba de concepto de una interfaz conversacional** destinada a asistir a los estudiantes en su planificación académica. Esta herramienta permitirá a los usuarios realizar consultas en lenguaje natural para obtener respuestas personalizadas sobre su carrera y su situación académica, así como recibir recomendaciones de asignaturas.

3.2 Requisitos

En la Tabla 3.1 se presenta un resumen de los requisitos funcionales y no funcionales del sistema, los cuales se detallan a continuación junto con los criterios de aceptación.

ID	Tipo	Nombre	Rol / Origen
RF-GEN-01	Funcional	Gestión de cuenta de usuario	General
RF-ADM-01	Funcional	Gestión de archivos públicos	Administrador
RF-ADM-02	Funcional	Gestión de preguntas frecuentes (FAQ)	Administrador
RF-ADM-03	Funcional	Análisis de uso del chatbot	Administrador
RF-ADM-04	Funcional	Gestión de modelos de lenguaje (LLM)	Administrador
RF-ADM-05	Funcional	Gestión de materiales por asignatura	Administrador
RF-EST-01	Funcional	Gestión de archivos privados del estudiante	Estudiante
RF-EST-02	Funcional	Sistema de valoración de respuestas	Estudiante
RF-EST-03	Funcional	Envío de comentarios a administradores	Estudiante
RF-EST-04	Funcional	Gestión del plan de estudio	Estudiante
RNF-GEN-01	No funcional	Manejo seguro de datos personales	General
RNF-GEN-02	No funcional	Replicabilidad e implementación en otras instituciones	General
RNF-GEN-03	No funcional	Almacenamiento local de datos sensibles	General
RNF-GEN-04	No funcional	Integración mediante APIs y microservicios	General
RNF-GEN-05	No funcional	Rendimiento y tiempos de respuesta	General
RNF-GEN-06	No funcional	Capacidad de ejecución local	General
RNF-GEN-07	No funcional	Disponibilidad	General
RNF-GEN-08	No funcional	Control de acceso por roles	General
RNF-EST-01	No funcional	Consultas mediante LLM con RAG	Estudiante

Tabla 3.1: Matriz de trazabilidad de requisitos

3.2.1 Requisitos funcionales

Requisitos Generales

- **RF-GEN-01 – Gestión de cuenta de usuario:** El sistema deberá permitir a cualquier usuario registrarse, iniciar sesión, cerrar sesión y recuperar sus credenciales mediante mecanismos de autenticación seguros.

Criterios de aceptación:

- El registro solicitará únicamente los datos mínimos necesarios.
- La recuperación de credenciales se realizará mediante un canal seguro (por ejemplo, correo electrónico).

- Solo los usuarios autenticados podrán acceder a las funcionalidades personalizadas.

Requisitos del Rol Administrador

- **RF-ADM-01 – Gestión de archivos públicos:** El sistema deberá permitir al administrador cargar, modificar, eliminar y organizar archivos institucionales de acceso público.

Criterios de aceptación:

- Los archivos serán visibles para todos los usuarios autorizados.
- El sistema deberá registrar las versiones o los cambios realizados en los archivos.

- **RF-ADM-02 – Gestión de preguntas frecuentes (FAQ):** El sistema deberá proporcionar una interfaz para que el administrador cree, actualice y elimine preguntas frecuentes utilizadas por el chatbot.

Criterios de aceptación:

- Los cambios en la base de datos de FAQ deberán reflejarse de inmediato en el chatbot.
- El administrador deberá poder gestionar preguntas y respuestas mediante una interfaz dedicada.

- **RF-ADM-03 – Análisis de uso del chatbot:** El sistema deberá recopilar, procesar y presentar datos estadísticos sobre el uso de forma anónima.

Criterios de aceptación:

- Los datos recopilados deberán estar anonimizados antes de cualquier procesamiento.
- Los reportes incluirán métricas básicas, como la frecuencia de uso, los temas consultados y la satisfacción, cuando corresponda.

- **RF-ADM-04 – Gestión de LLMs:** El sistema deberá permitir al administrador cargar, actualizar, activar o reemplazar los LLMs del chatbot.

Criterios de aceptación:

- Solo un modelo podrá estar activo en el entorno de producción.
- El sistema deberá validar la compatibilidad y el estado del modelo antes de su activación.

- **RF-ADM-05 – Gestión de materiales por asignatura:** El sistema deberá permitir al administrador cargar, actualizar y eliminar materiales asociados a las asignaturas del plan de estudios. **Criterios de aceptación:**
 - Los materiales cargados deberán incorporarse a la base de conocimiento del asistente.
 - El administrador podrá gestionar los materiales de cualquier asignatura del sistema.

Requisitos del Rol Estudiante

- **RF-EST-01 – Gestión de archivos privados:** El sistema deberá permitir a cada estudiante cargar, organizar y eliminar sus archivos privados, garantizando el acceso exclusivo de cada estudiante.

Criterios de aceptación:

- Solo el estudiante propietario podrá acceder a sus archivos.
 - Los archivos privados solo serán utilizados por el asistente para personalizar las respuestas.
- **RF-EST-02 – Sistema de valoración de respuestas:** El sistema deberá permitir al estudiante calificar la utilidad de las respuestas del chatbot mediante un mecanismo de valoración.

Criterios de aceptación:

- La valoración deberá registrarse sin almacenar información sensible asociada.
 - Las valoraciones deberán utilizarse para mejorar el comportamiento del asistente.
- **RF-EST-03 – Envío de comentarios a administradores:** El sistema deberá permitir que el estudiante envíe comentarios o sugerencias mediante un formulario estructurado.

Criterios de aceptación:

- El formulario deberá incluir la categoría, la descripción y el nivel de prioridad.
 - Los envíos deberán registrarse para su posterior revisión por los administradores.

- **RF-EST-04 – Gestión del plan de estudio:** El sistema deberá permitir al estudiante seleccionar, registrar y modificar las asignaturas que planea cursar en futuros semestres.

Criterios de aceptación:

- Las asignaturas seleccionadas deberán estar disponibles para el asistente conversacional.
- El chatbot deberá utilizar esta información para generar recomendaciones personalizadas.

3.2.2 Requisitos no funcionales

Requisitos Generales

- **RNF-GEN-01 – Manejo seguro de datos personales:** El sistema deberá administrar la información personal de los usuarios de manera segura, en cumplimiento de las normativas vigentes de protección de datos. Esto incluye el almacenamiento cifrado de datos sensibles y la posibilidad de que el usuario modifique o elimine su información.

Criterios de aceptación:

- Los datos sensibles deberán almacenarse mediante cifrado en reposo (AES-256 u otro estándar equivalente).
- El sistema deberá permitir que el usuario solicite la modificación o la eliminación de sus datos personales.
- El acceso a datos personales deberá estar auditado y limitado según el rol del usuario.

- **RNF-GEN-02 – Replicabilidad e implementación en otras instituciones:** El sistema deberá ser modular y configurable, permitiendo su despliegue en diferentes instituciones educativas sin requerir cambios estructurales significativos.

Criterios de aceptación:

- La configuración institucional deberá estar desacoplada del código fuente.
- La adaptación a otra institución no requerirá modificaciones en la arquitectura del sistema.

- Las dependencias externas deberán estar documentadas y parametrizadas.
- **RNF-GEN-03 – Almacenamiento local de datos sensibles:** El sistema deberá almacenar información sensible localmente o en servidores controlados por la institución, evitando el envío de datos personales a servicios externos.

Criterios de aceptación:

- Los datos sensibles no deberán ser transferidos a terceros sin autorización institucional.
- El almacenamiento deberá realizarse en infraestructura local o en servidores bajo el control directo de la institución.
- **RNF-GEN-04 – Integración mediante APIs y arquitectura de microservicios:** El sistema deberá permitir la integración con otros servicios institucionales mediante APIs, siguiendo una arquitectura basada en microservicios.

Criterios de aceptación:

- Las APIs deberán cumplir con los estándares de REST y contar con documentación en OpenAPI/Swagger.
- Los módulos del sistema deberán funcionar de forma desacoplada.
- El sistema deberá soportar la incorporación de nuevos servicios sin afectar los componentes existentes.
- **RNF-GEN-05 – Rendimiento y tiempos de respuesta:** El sistema deberá responder consultas simples y operaciones del asistente de forma fluida y ejecutar operaciones complejas en tiempos razonables bajo condiciones normales de carga.

Criterios de aceptación:

- Las consultas simples deberán responderse en menos de 2 segundos.
- Las operaciones complejas no deberán exceder de 5 a 8 segundos bajo la carga estándar.
- Los tiempos de respuesta no deberán degradarse significativamente con una carga concurrente moderada.
- **RNF-GEN-06 – Capacidad de ejecución local:** El sistema deberá ejecutarse correctamente en un entorno local controlado, manteniendo tiempos de respuesta aceptables y sin fallos recurrentes durante su uso normal.

Criterios de aceptación:

- El sistema deberá responder consultas simples y complejas en tiempos aceptables en el entorno de hardware definido para la prueba de concepto.
 - No deberán producirse errores por ejecución simultánea de procesos internos del sistema.
 - En pruebas locales con uso acotado, el sistema deberá mantenerse estable.
- **RNF-GEN-07 – Disponibilidad:** El sistema deberá mantenerse disponible para estudiantes y coordinadores durante la mayor parte del período lectivo, evitando caídas frecuentes.

Criterios de aceptación:

- La disponibilidad del sistema deberá ser igual o superior al 99 % durante los períodos académicos.
 - En caso de caída, la recuperación del servicio no deberá exceder los tiempos operativos aceptables establecidos por la institución.
- **RNF-GEN-08 – Control de acceso por roles:** El sistema deberá implementar permisos diferenciados para cada tipo de usuario, limitando las acciones según su rol.

Criterios de aceptación:

- El sistema deberá autenticar y autorizar a cada usuario según su rol.
- Las acciones no permitidas deberán generar mensajes de acceso denegado.
- Las reglas de acceso deberán documentarse y mantenerse consistentes en todas las interfaces.

Requisitos del Rol Estudiante

- **RNF-EST-01 – Consultas mediante LLM con RAG:** El sistema deberá permitir que los estudiantes realicen consultas al asistente conversacional mediante un modelo de lenguaje basado en la técnica de Retrieval-Augmented Generation (RAG).

Criterios de aceptación:

- El asistente deberá combinar las respuestas del LLM con la información de la base de conocimiento institucional.

- Las fuentes utilizadas en el proceso RAG deberán ser actualizadas y gestionadas por los roles autorizados.
- El asistente deberá evitar las alucinaciones mediante la recuperación y la verificación del contenido cuando sea posible.

4 Diseño e implementación de la solución

En este capítulo se presentan la metodología de trabajo adoptada, las etapas del desarrollo, la implementación concreta de la solución que se encuentra en [5] y la evaluación de la prueba de concepto.

4.1 Metodología de trabajo adoptada

La metodología seleccionada para el presente proyecto se enmarca en la investigación tecnológica aplicada, orientada al diseño y desarrollo de una prueba de concepto funcional. El enfoque de trabajo es de carácter iterativo e incremental, lo que ha permitido ajustar la arquitectura del sistema a medida que se han identificado limitaciones técnicas en el procesamiento del lenguaje natural y en el acceso a datos académicos.

El marco metodológico se fundamenta en los siguientes pilares:

- **Enfoque centrado en la privacidad y la soberanía de los datos:** A diferencia de las soluciones convencionales basadas en servicios externos, la metodología adoptada prioriza el procesamiento local. Esto implica una decisión técnica de no utilizar APIs de terceros y de optar por la ejecución local de LLMs en entornos como Ollama, garantizando que la información sensible de los estudiantes permanezca dentro de la infraestructura institucional.
- **Arquitectura evolutiva de agentes inteligentes:** Inicialmente, el proyecto se concibió con un flujo de Generación Aumentada por Recuperación (RAG) tradicional. Sin embargo, ante la complejidad de las consultas académicas, la metodología evolucionó hacia la implementación de un sistema

de agentes basado en el patrón ReAct (*Reasoning + Acting*). Este enfoque permite que el sistema no solo recupere información, sino que también razone sobre qué herramientas invocar para ofrecer respuestas más precisas y deterministas.

- **Modularización funcional:** El desarrollo se organiza en una arquitectura de tres capas: Frontend, Backend e Infraestructura. Esta separación permite trabajar de forma independiente en la interfaz conversacional, la lógica de negocio y la orquestación de la base de datos vectorial, lo que facilita la escalabilidad del sistema.
- **Validación mediante caso de aplicación:** La metodología contempla una fase final de evaluación en la que se utiliza el contexto real de la Licenciatura en Sistemas de la Universidad ORT Uruguay. Esto permite contrastar la eficacia de los LLMs con los planes de estudio y los reglamentos vigentes, asegurando la utilidad práctica de la herramienta.

Esta metodología asegura un equilibrio entre la innovación tecnológica que proponen los LLMs modernos y la rigurosidad necesaria para el manejo de la información académica institucional.

4.2 Etapas del desarrollo

El desarrollo de la prueba de concepto se estructuró en fases incrementales, con un enfoque especial en la resolución de conflictos arquitectónicos derivados de la integración de LLMs con datos institucionales. El proceso abarcó aproximadamente **26 semanas** y las etapas se detallan a continuación:

1. **Investigación y selección del stack tecnológico (semanas 1–2):** Se evaluaron LLMs de código abierto y herramientas de orquestación. Se definió el uso de **Ollama** para garantizar la soberanía de los datos y se seleccionaron **FastAPI** y **React** para asegurar un alto rendimiento en la comunicación asíncrona entre el agente y la interfaz.
2. **Configuración de infraestructura y persistencia (semanas 3–9):** Se implementó la arquitectura contenerizada mediante **Docker Compose**, integrando **MongoDB** para el historial de conversaciones y los metadatos y **ChromaDB** como motor de búsqueda vectorial.

3. **Desarrollo del Dashboard de Gestión (semanas 10–11):** Se desarrolló una interfaz específica para administradores y funcionarios de la facultad. Esta herramienta permite la carga centralizada de documentos institucionales (planes de estudio, reglamentos y materiales), automatizando el proceso de ingesta y de vectorización en la base de conocimiento.
4. **Evolución de RAG hacia Herramientas de Agente (semanas 12–14):** Durante la integración, se detectó un conflicto técnico fundamental: el flujo de un sistema RAG tradicional chocaba con la lógica de toma de decisiones del agente ReAct, lo que impedía el funcionamiento simultáneo de ambos. Para resolverlo, se rediseñó la arquitectura de modo que el agente asumiera las responsabilidades del RAG. Se implementaron herramientas específicas (*tools*) que permiten al agente buscar fragmentos semánticos o leer archivos completos de forma controlada, unificando el razonamiento y la recuperación de información en un único flujo orquestado por **LangGraph**.
5. **Implementación de la lógica académica y de herramientas deterministas (semanas 15–17):** Se desarrollaron las funciones en Python que permiten al agente consultar la escolaridad del estudiante y validar prerrequisitos con exactitud. Esto evitó delegar cálculos lógicos en el modelo de lenguaje, lo que redujo drásticamente el riesgo de alucinaciones en las recomendaciones curriculares.
6. **Ciclo de pruebas internas y validación aislada (semanas 18–26):** En la fase final, se llevaron a cabo pruebas exhaustivas por parte del equipo de desarrollo para depurar el comportamiento del agente. Asimismo, se realizaron pruebas de uso aisladas con personas ajenas al proyecto para evaluar la usabilidad de la interfaz conversacional y la pertinencia de las respuestas generadas antes de la validación técnica definitiva.

4.3 Solución del problema

Para abordar el problema planteado, se empleó una arquitectura de **Generación Aumentada por Recuperación (RAG)**. Esta arquitectura combina las capacidades de generación de LLMs con la precisión de una base de conocimiento externa.

Específicamente, el sistema utiliza un conjunto de LLMs de código abierto, ejecutados localmente mediante **Ollama**, para interpretar las preguntas y generar respuestas. Para evitar alucinaciones y asegurar que las respuestas sean correctas

y estén actualizadas, el LLM se amplía con contexto recuperado a partir de una base de datos vectorial. Esta base de datos contiene información institucional verificable, como planes de estudio, reglamentos y patrones identificados mediante el **análisis curricular** de trayectorias estudiantiles previas. El stack tecnológico se complementa con **LangChain** para orquestar el flujo RAG y con **FastAPI** como backend para exponer la interfaz.

La solución se organiza en una arquitectura de tres capas:

- **Backend:** una API REST desarrollada en Python con FastAPI, que implementa la lógica de negocio, el pipeline RAG y la integración con el modelo de lenguaje.
- **Frontend:** una aplicación web de una sola página (SPA) construida en React, que expone la interfaz conversacional al estudiante y orquesta la interacción con la API.
- **Infraestructura / DevOps:** un conjunto de servicios dockerizados (backend, frontend, base de datos MongoDB, Ollama y ChromaDB) coordinados mediante Docker Compose, con volúmenes persistentes para modelos, vectores y datos de usuario.

A nivel funcional, el sistema cumple con todos los requisitos funcionales y no funcionales definidos en la Sección 3.2.

4.3.1 Backend

El backend se implementó en **Python** utilizando el framework **FastAPI**, elegido por su buen rendimiento, soporte nativo para operaciones asíncronas y su integración con el ecosistema de librerías de inteligencia artificial.

La API expone endpoints REST que permiten:

- gestionar usuarios y sus perfiles;
- administrar documentos y la base de conocimiento;
- ejecutar consultas RAG;
- registrar y consultar el historial de conversaciones;
- recolectar comentarios sobre las respuestas del asistente.

Los componentes principales del backend son:

- **FastAPI + Uvicorn:** FastAPI define los endpoints y los modelos de datos (requests/responses), mientras que Uvicorn actúa como servidor ASGI para atender las peticiones HTTP de forma eficiente.
- **MongoDB:** se utiliza como base de datos principal para almacenar:
 - usuarios (incluyendo el identificador sub de Auth0 y el rol: estudiante o administrador);
 - conversaciones y mensajes;
 - metadatos de los archivos cargados;
 - feedback sobre las respuestas del asistente.

El sistema implementa multi-tenancy: los estudiantes pueden subir archivos privados (solo visibles para ellos) y los administradores, archivos públicos (visibles para todos los usuarios). ChromaDB filtra automáticamente los resultados de búsqueda según estos permisos, garantizando que cada usuario solo acceda a la información autorizada.

- **ChromaDB** (base de datos vectorial): almacena los embeddings de los fragmentos de texto extraídos de los documentos. Esto permite que, ante una pregunta del usuario, el sistema recupere rápidamente los fragmentos más relevantes mediante una búsqueda por similitud. El agente accede a esta base de datos mediante herramientas, en lugar de que el contexto se inyecte automáticamente en el prompt.
- **LangGraph:** se utiliza como orquestador principal del sistema de agentes ReAct. LangGraph permite definir grafos de estado que controlan el flujo de ejecución del agente, incluyendo:
 - Análisis de la consulta del usuario.
 - Selección automática de las herramientas a invocar.
 - Ejecución de herramientas (búsqueda en documentos, consulta de datos académicos, etc.).
 - Procesamiento de resultados y generación de respuesta mediante el LLM
 - Devolución de una respuesta final al usuario.

El sistema mantiene compatibilidad con el flujo RAG tradicional mediante LangChain, que encapsula los pasos clásicos de RAG (embedding, consulta a ChromaDB, construcción de prompt enriquecido), pero este flujo no se utiliza en el chat normal cuando el agente está habilitado

- **Ollama + modelo de lenguaje:** el modelo de lenguaje (LLM) se ejecuta localmente mediante Ollama. La API del backend se comunica con Ollama vía HTTP, enviando el prompt enriquecido y recibiendo como respuesta el texto generado por el modelo. Esto permite mantener la privacidad de los datos y eliminar la dependencia de APIs de terceros en la nube.
- **Auth0:** se encarga de la autenticación de los usuarios. El backend valida el token JWT en cada petición y extrae de él la identidad del usuario y su rol, aplicando las reglas de autorización correspondientes (por ejemplo, restringir determinados endpoints solo a administradores).

La Figura 4.1 presenta el diagrama de arquitectura del backend, Mostrando la relación entre los componentes principales y el flujo de datos desde la petición del usuario hasta la generación de la respuesta.

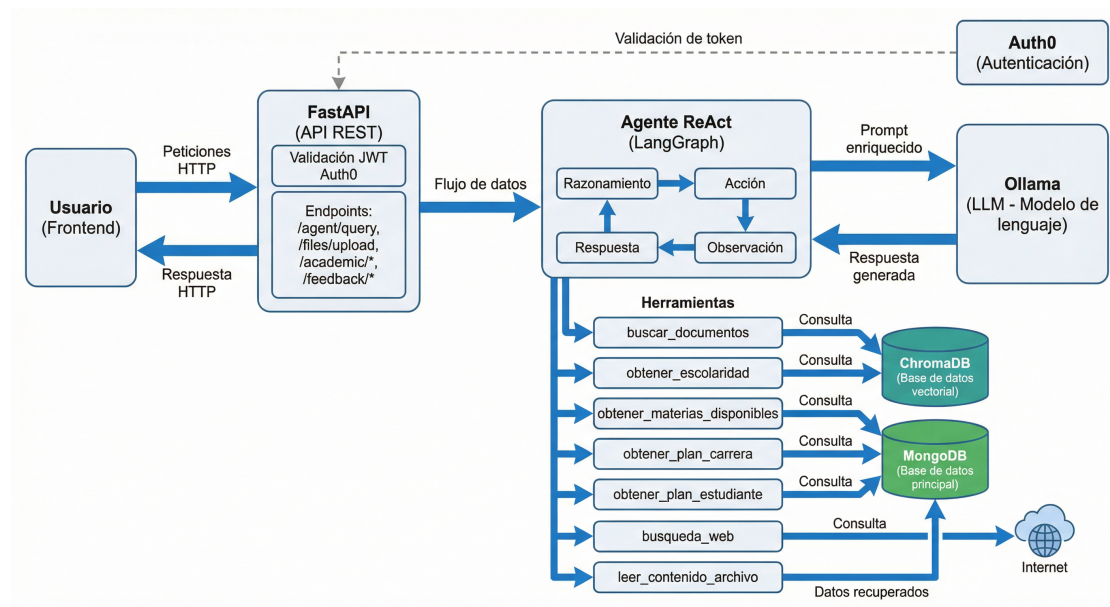


Figura 4.1: Diagrama de arquitectura del backend.

Se ilustra la interacción entre FastAPI (API REST), el agente ReAct (Lang-Graph), las herramientas del agente (search documents, get student schooling, get available courses, etc.), la base de datos vectorial (ChromaDB), la base de datos principal (MongoDB), el modelo de lenguaje (Ollama) y el servicio de autenticación (Auth0). Las flechas indican el flujo de datos: el usuario envía una consulta al

endpoint de FastAPI, que la delega al agente ReAct; el agente razona sobre qué herramientas invocar, ejecuta las herramientas necesarias (que consultan ChromaDB o MongoDB) y, finalmente, envía el prompt enriquecido al LLM vía Ollama para generar la respuesta.

4.3.1.1 API

A continuación se describe el flujo completo que ocurre en la API cuando un estudiante conversa con el asistente:

1. Autenticación del usuario

- El estudiante accede al frontend y selecciona la opción de iniciar sesión.
- El navegador redirige al sitio de Auth0, donde el usuario se registra o inicia sesión.
- Tras el éxito del proceso, Auth0 devuelve al frontend un token JWT de acceso.

2. Carga e indexación de documentos

- Desde la sección de archivos, el usuario (o un administrador) selecciona los documentos para subirlos.
- El frontend envía una petición POST a `/files/upload` con el archivo adjunto y el token JWT.
- El backend:
 - almacena el archivo en MongoDB GridFS (sistema de almacenamiento de archivos grandes de MongoDB), asociando metadatos como usuario propietario y flag (público/privado);
 - extrae el texto (según el tipo de archivo: PDF, Word, Excel, texto plano, etc.);
 - divide el contenido en fragmentos (chunks);
 - genera los embeddings de cada fragmento y los guarda en ChromaDB, junto con el texto original y los metadatos (nombre de archivo, usuario propietario, si es público o privado).

3. Inicio de una conversación

- En la vista de chat, el estudiante escribe una pregunta en lenguaje natural.

- El frontend envía una petición POST a `/agent/query` (o `/rag/query` como fallback) al backend con:
 - el texto de la pregunta;
 - parámetros adicionales (idioma de la respuesta, cantidad de fragmentos a recuperar, identificador de conversación si ya existe);
 - el token JWT en la cabecera `Authorization` con el valor `Bearer` seguido del token.

4. Procesamiento mediante Agente ReAct (flujo principal)

- El sistema utiliza el agente ReAct como flujo principal:
 - a) El agente analiza la consulta del usuario y determina qué herramientas debe invocar.
 - b) Si requiere buscar en documentos, invoca `'search_documents'`, que:
 - genera un embedding de la pregunta;
 - consulta ChromaDB para obtener los chunks más relevantes;
 - devuelve los resultados al agente.
 - c) Si necesita más contexto, el agente puede invocar `'read_file_content'` para leer el archivo completo.
 - d) El agente procesa la información recuperada y genera una respuesta mediante el LLM.

5. Llamada al modelo de lenguaje (LLM)

- El agente envía el prompt enriquecido (con el contexto de herramientas, si es necesario) a Ollama mediante su API local, indicando el modelo seleccionado.
- Ollama ejecuta el modelo y devuelve una respuesta generada.
- El backend recibe esta respuesta, la limpia o formatea si es necesario y construye un objeto de respuesta que incluye:
 - el texto final a mostrar;
 - el listado de fuentes (nombres de archivo de donde se extrajo el contexto o herramientas utilizadas);
 - la cantidad de fragmentos utilizados;
 - el identificador de la conversación.

6. Persistencia del historial de conversación

- Antes de responder al frontend, el backend guarda en MongoDB:

- la pregunta del usuario como un mensaje;
 - la respuesta generada por el LLM;
 - metadatos (fecha, tiempo de respuesta, documentos utilizados, usuario, etc.).
- Si es la primera interacción, se crea una nueva conversación; si no, se agrega el mensaje a una conversación ya existente.

7. Respuesta al frontend

- El backend envía al frontend un JSON con el contenido de la respuesta y la información de contexto.
- El frontend actualiza la interfaz de chat, mostrando:
 - el mensaje del usuario;
 - la respuesta del asistente;
 - las fuentes utilizadas (por ejemplo, los nombres de los documentos relevantes).

8. Registro de feedback

- El estudiante puede marcar la respuesta como útil o no útil y agregar un comentario opcional.
- Esto genera una petición POST a `/feedback/message`, que almacena en MongoDB la evaluación del usuario, vinculada al mensaje y a la conversación.
- Posteriormente, los administradores pueden consultar estadísticas agregadas y resúmenes de feedback mediante endpoints específicos de administración.

4.3.1.2 Desafíos técnicos: instrucciones al LLM y solución con Instructor

Durante el desarrollo del sistema, se enfrentó un desafío crítico relacionado con la capacidad del modelo de lenguaje para seguir instrucciones complejas, especialmente cuando se requerían respuestas estructuradas con artefactos (código, tablas, diagramas).

Problema identificado.

Inicialmente, el sistema solicitaba al LLM que generara respuestas en formato JSON estructurado, incluyendo artefactos como tablas o código. Sin embargo, se observó que los modelos locales ejecutados mediante Ollama (como gemma3 o llama2) con frecuencia ignoraban estas instrucciones, generando texto plano o Markdown inconsistente en lugar del formato JSON solicitado. Esto generaba inconsistencias en las respuestas y dificultaba el análisis confiable de los resultados.

Ejemplo del problema.

Cuando se solicitaba al LLM que generara una tabla estructurada en formato JSON con artifacts, el modelo respondía con Markdown simple, ignorando por completo las instrucciones de formato estructurado.

Solución implementada: Instructor.

Para abordar este problema, se implementó la librería Instructor, una herramienta de Python que permite obtener respuestas estructuradas de LLMs mediante modelos de Pydantic. Instructor funciona “parcheando” el cliente de OpenAI (compatible con Ollama) para forzar respuestas en formato JSON válido.

Limitaciones encontradas.

Durante las pruebas se identificaron limitaciones: los modelos locales de Ollama no siempre respetaban el formato JSON estructurado, incluso con Instructor, lo que generaba errores de parsing frecuentes. Además, Instructor funciona mejor con modelos grandes como GPT-4 o Claude, mientras que los modelos locales tienen capacidades más limitadas. Por estas razones, aunque el código de Instructor permanece en el proyecto, actualmente no se utiliza en el flujo principal de generación de respuestas.

Solución final: enfoque “Markdown-first”.

Ante las limitaciones encontradas con Instructor y con los modelos locales, se adoptó un enfoque alternativo denominado “Markdown-first”. En lugar de forzar al LLM a generar JSON estructurado, se permite que el modelo genere Markdown natural (lo que hace mejor) y el frontend extrae los artefactos del Markdown.

Este enfoque resultó más confiable porque:

- El LLM genera Markdown natural, lo cual es su fortaleza.
- El parsing en el frontend es más robusto y controlable.
- No depende de que el LLM genere un JSON perfecto.
- Funciona de forma consistente con cualquier modelo LLM.

Esta experiencia ilustra la importancia de adaptar las soluciones técnicas a las limitaciones reales de los modelos disponibles, priorizando la confiabilidad por encima de la complejidad teórica. El código de Instructor se mantiene en el proyecto como referencia y para posibles usos futuros con modelos más capaces.

4.3.1.3 Sistema de agentes ReAct

El sistema implementa un agente inteligente basado en el patrón ReAct (Reasoning + Acting) que utiliza LangGraph. Este agente permite que el modelo de lenguaje seleccione automáticamente qué herramientas (tools) utilizar en función de la consulta del usuario, lo que mejora significativamente la capacidad del sistema para responder consultas complejas sobre planificación académica.

Arquitectura del agente.

El agente ReAct utiliza `create_react_agent()` de LangGraph, que implementa un flujo de trabajo en el que el LLM analiza la consulta del usuario y decide qué herramientas invocar. El patrón ReAct sigue un ciclo de razonamiento (reasoning), acción (acting) y observación (observation) hasta completar la respuesta.

Herramientas disponibles.

El agente tiene acceso a las siguientes herramientas que puede invocar automáticamente:

- **search_documents**: busca en los documentos subidos por el usuario.
- **get_student_schooling**: obtiene la escolaridad del estudiante, incluyendo materias completadas, en curso, GPA y estadísticas académicas.
- **get_completed_courses**: obtiene las materias completadas del estudiante, junto con sus calificaciones y su GPA.
- **get_current_courses**: obtiene las materias en curso del estudiante.
- **get_course_recommendations**: obtiene recomendaciones personalizadas de materias utilizando los algoritmos de ML desarrollados en el proyecto Study Buddy [13] (Random Forest, Sequential Pattern Mining y Pattern Mining), ordenadas por probabilidad de aprobación y por relevancia académica.
- **get_available_courses**: obtiene las materias disponibles para inscripción, validando los prerrequisitos de forma determinista en Python.
- **get_degree_curriculum**: obtiene el plan de estudios completo de una carrera.
- **get_student_plan**: obtiene el plan de estudios personalizado guardado.
- **create_study_plan**: genera un nuevo plan de estudios para semestres futuros y lo guarda en la base de datos.
- **web_search**: realiza búsquedas en la web para obtener información actualizada.

Flujo de ejecución.

1. El usuario realiza una consulta en lenguaje natural (p. ej., ¿Qué materias estoy cursando?).
2. El LLM analiza la intención de la consulta y selecciona la herramienta apropiada (por ejemplo, `get_completed_courses` o `get_course_recommendations`).
3. La herramienta se ejecuta mediante `HTTPToolExecutor`, que realiza una petición HTTP al servicio correspondiente del backend.

4. El LLM recibe los resultados de la herramienta y genera una respuesta en lenguaje natural en el mismo idioma que la consulta.
5. La respuesta se devuelve al usuario junto con información sobre las fuentes utilizadas.

Ventajas del enfoque ReAct.

- **Selección inteligente de herramientas:** el LLM decide qué herramienta usar a partir de la descripción de cada una y de la intención de la consulta.
- **Validación determinista:** las validaciones críticas (como prerequisites) se realizan en Python, no en el LLM, lo que garantiza su confiabilidad.
- **Soporte multilingüe:** el agente responde automáticamente en el mismo idioma que el de la consulta del usuario.
- **Extensibilidad:** agregar nuevas herramientas es sencillo.

4.3.1.4 Sistema académico

El sistema incluye un módulo completo para la gestión de la información académica, lo que permite al agente acceder a datos estructurados sobre carreras, materias, escolaridad de los estudiantes y planes de estudio.

Modelos de datos.

El sistema define los siguientes modelos en MongoDB:

- **DegreeInDB:** representa una carrera universitaria, incluyendo nombre, créditos totales, duración en semestres y descripción.
- **DegreeSubjectInDB:** representa una materia dentro de una carrera, con información sobre créditos, prerequisites, corequisites y el semestre sugerido.
- **StudentSchoolingInDB:** representa la escolaridad (historial académico) de un estudiante, incluyendo materias completadas, en curso, GPA y estadísticas.
- **StudentPlanInDB:** representa el plan de estudios futuro del estudiante, con materias planificadas por semestre.

Endpoints del sistema académico.

El backend expone endpoints RESTful bajo el prefijo `/academic` que permiten:

- Consultar carreras disponibles (`GET/academic/degrees`).
- Obtener el plan de estudios de una carrera (`GET/academic/degrees/<degree_id>/subjects`).
- Consultar la escolaridad del estudiante (`GET/academic/schooling`).
- Obtener recomendaciones de materias basadas en ML (`GET /academic/students/me/recommendations/{degree_id}`), con parámetro `algorithm: random_forest, spm` o `all`.
- Obtener las materias disponibles para la inscripción (`GET/academic/available-courses`).
- Gestionar el plan de estudios del estudiante (`GET/academic/plan` y `POST/academic/plan`).

Integración con el agente.

El agente ReAct utiliza estas herramientas para responder consultas como:

- ¿Qué materias estoy cursando? → invoca `get_current_courses` y `get_completed_courses`.
- ¿En qué materias puedo inscribirme el próximo semestre? → invoca `get_available_courses`.
- ¿Qué materias me recomendás para el próximo semestre? → invoca `get_course_recommendations` y opcionalmente `create_study_plan`.
- ¿Cuál es el plan de estudios de Ingeniería en Sistemas? → invoca `get_degree_curriculum`.
- Muéstrame mi GPA → invoca `get_completed_courses`.

4.3.1.5 Evolución del sistema y decisiones de implementación

Durante el desarrollo del proyecto, se tomaron varias decisiones arquitectónicas importantes que modificaron significativamente la implementación inicial. Esta sección documenta los cambios más relevantes.

Evolución de RAG a agentes.

El sistema inicialmente implementó una arquitectura RAG (Retrieval-Augmented Generation) para la lectura de archivos y la generación de respuestas. Sin embargo, al implementar el sistema de agentes para cumplir con los requerimientos adicionales, se encontró un conflicto fundamental: no podía utilizar ambos enfoques de manera efectiva simultáneamente. El RAG y los agentes competían por el control del flujo de procesamiento, lo que generaba inconsistencias en las respuestas.

La solución adoptada fue que el agente ReAct asumiera las responsabilidades del RAG. Aunque el código del RAG permanece implementado, en el uso normal del chat no se invoca directamente. En su lugar, el agente utiliza herramientas (por ejemplo, `search_documents` y `read_file_content`) que, internamente, acceden a la misma base de datos vectorial (ChromaDB) que el RAG, pero de forma controlada por el agente.

El flujo actual funciona de la siguiente manera:

1. El agente recibe una consulta del usuario.
2. Si necesita buscar información en documentos, invoca `search_documents`, que realiza una búsqueda semántica en ChromaDB.
3. Una vez obtenidos los chunks relevantes, el agente puede invocar `read_file_content` para leer el archivo completo si necesita más contexto.
4. El agente procesa la información recuperada y genera una respuesta.

Esta arquitectura no es estrictamente RAG, ya que el agente tiene control sobre cuándo y cómo acceder a los documentos, en lugar de que el contexto se inyecte automáticamente en el prompt del LLM. El RAG original queda disponible como fallback o para casos de uso específicos que no requieran las capacidades del agente.

Selección y cambio de modelos.

Durante el desarrollo se probaron varios LLMs:

- Modelos muy pequeños (como TinyLlama) devolvían respuestas incorrectas o incoherentes debido a su limitada capacidad de procesamiento.
- Modelo de Google (Gemma): se utilizó durante gran parte del desarrollo y se mantiene como modelo principal.

Aumento del contexto de los modelos.

Inicialmente, el sistema utilizaba un contexto de cierta cantidad de caracteres, pero durante el desarrollo se identificó que este límite resultaba insuficiente, especialmente en conversaciones multimensaje, donde el historial de mensajes previos se incluía en el contexto.

Se investigó el impacto de incrementar el contexto y se determinó que, contrariamente a lo esperado, el costo computacional no aumentó significativamente. El sistema se configuró para utilizar muchos más caracteres de contexto, lo que permitió:

- Reducir los problemas de alucinación al proporcionar más contexto al modelo.
- Soportar conversaciones más largas sin perder coherencia.
- Mejorar la calidad de las respuestas a consultas complejas.

Problema con Instructor y migración a ReAct.

Como se documenta en la Sección 4.3.1.2, inicialmente se intentó utilizar la librería Instructor para forzar respuestas estructuradas del LLM. Sin embargo, este enfoque resultó problemático por varias razones:

1. **Acumulación de instrucciones:** para hacer el sistema determinista y evitar errores en casos borde, se necesitaba agregar instrucciones específicas para cada caso problemático. Esto llevó a que se acumularan más de 400 líneas de instrucciones en el prompt.

2. **Sobrecarga del modelo:** los modelos locales no pueden procesar eficientemente tal cantidad de instrucciones, especialmente cuando se combinan con el contexto de la consulta y con los datos recuperados.
3. **Casos de borde infinitos:** los usuarios pueden formular consultas de manera impredecible, lo que requeriría instrucciones para cada una de las posibles variantes.

La migración a ReAct resolvió estos problemas al reducir significativamente las instrucciones necesarias y permitir que el agente tomara decisiones más simples y directas sobre qué herramienta utilizar. Los caminos de decisión son más cortos y, por lo tanto, más confiables.

4.3.2 Frontend

El frontend se desarrolló como una Single Page Application utilizando **React** y **Vite** como herramienta de construcción. La aplicación se organiza en páginas y componentes que corresponden a los principales casos de uso del sistema.

Los módulos más relevantes son:

- **Módulo de autenticación:** se integra con el SDK de Auth0 para React. Gestiona el inicio y el cierre de sesión y expone el token JWT al resto de la aplicación.
- **Módulo de chat:** implementa la interfaz conversacional. Permite:
 - escribir y enviar mensajes;
 - visualizar respuestas del asistente;
 - mostrar indicadores de carga mientras la API procesa la consulta;
 - mostrar las fuentes utilizadas en cada respuesta.
- **Módulo de archivos:** permite subir, listar y eliminar documentos accesibles para el usuario.
- **Historial de conversaciones:** lista las conversaciones previas del estudiante y permite reabrirlos para continuar desde donde se dejaron.
- **Módulo de administración** (visible solo para el rol de administrador): muestra estadísticas de uso y comentarios agregados sobre el sistema.

Toda comunicación con el backend se realiza mediante peticiones HTTP a la API de FastAPI, adjuntando siempre el token JWT en la cabecera Authorization. De esta manera, la lógica de autorización se centraliza en el backend y el frontend se mantiene relativamente ligero.

En la Figura 4.2 se muestra la estructura de la aplicación React, organizada en sus módulos principales: Autenticación (Auth0 SDK), Chat (interfaz conversacional con parsing de Markdown y renderizado de artefactos), Archivos (gestión de documentos), Historial (conversaciones previas) y Administración (estadísticas y feedback). Las flechas indican la comunicación HTTP con la API REST del backend (FastAPI), donde cada petición incluye el token JWT en la cabecera de autorización. Vite actúa como herramienta de construcción y Nginx como servidor web en producción.

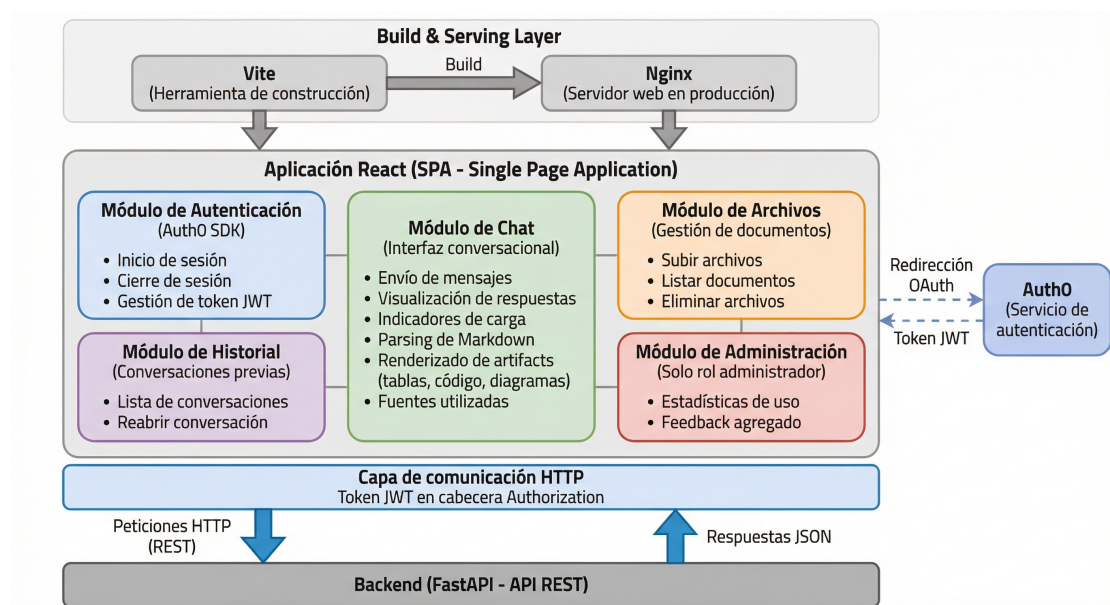


Figura 4.2: Diagrama de arquitectura del frontend.

4.3.3 DevOps

Para facilitar la ejecución y despliegue de la solución, se adoptó una estrategia basada en **Docker Compose**, empaquetando cada componente principal en un contenedor independiente:

- **fastapi-app**: contiene el backend con FastAPI, LangChain, LangGraph y la lógica de RAG.

- **frontend**: sirve la aplicación de React mediante Nginx.
- **mongodb**: almacena datos de usuarios, conversaciones, archivos y comentarios.
- **chromadb-server**: almacena los datos de usuarios, conversaciones, archivos y comentarios.
- **agent-api**: API separada que expone herramientas especializadas para el agente inteligente (búsqueda web, herramientas académicas, etc.).
- **ollama**: ejecuta el modelo de lenguaje localmente.

Adicionalmente, se definieron volúmenes persistentes para:

- uploads: documentos cargados por los usuarios;
- chroma_db: índice vectorial de los documentos;
- ollama-models: modelos descargados por Ollama;
- mongo-data: datos de la base de datos de MongoDB.

La configuración sensible (credenciales de MongoDB, parámetros de Auth0, modelo de Ollama, etc.) se gestiona mediante un archivo `.env` en la raíz del proyecto. Esta arquitectura permite levantar todo el sistema con un solo comando (`docker compose up`), reproducir el entorno en distintas máquinas de desarrollo y, en una etapa posterior, migrar a un entorno de producción con cambios mínimos en la configuración.

4.3.4 Comparativa tecnológica

En la Tabla 4.1 se presenta un resumen de las principales tecnologías empleadas en el proyecto y el rol que desempeña cada una dentro de la arquitectura.

Tecnología	Categoría	Uso en el proyecto
Ollama + Llama 3	LLM	Ejecución local del modelo de lenguaje para interpretar consultas y generar respuestas en lenguaje natural.
ChromaDB	Base de datos vectorial	Almacenamiento y consulta de embeddings de documentos institucionales para la búsqueda semántica.
LangChain	Orquestación RAG	Coordinación del flujo RAG: procesamiento de consultas, recuperación de contexto y generación de respuestas.
FastAPI	Backend	Exposición de endpoints REST para la comunicación entre el frontend y los componentes de IA.
Uvicorn	Servidor ASGI	Servidor asíncrono de alto rendimiento para ejecutar la aplicación FastAPI.
React	Frontend	Interfaz de usuario web para la interacción conversacional con el estudiante.
Docker	Contenedorización	Empaquetado y despliegue de todos los componentes del sistema de forma aislada y reproducible.
MongoDB	Base de datos	Almacenamiento del historial de conversaciones y metadatos del sistema.
Auth0	Autenticación	Gestión de inicio de sesión y autenticación de usuarios.
Python	Lenguaje	Lenguaje principal del backend y de la lógica de IA, elegido por su ecosistema de librerías para la inteligencia artificial.

Tabla 4.1: Resumen de las tecnologías empleadas en el proyecto y su rol en la arquitectura.

4.4 Evaluación de la solución

En esta sección se describe la estrategia de evaluación aplicada para validar el correcto funcionamiento de la prueba de concepto, se presentan los resultados obtenidos a partir de las pruebas realizadas y se documentan las limitaciones identificadas durante el proceso.

4.4.1 Estrategia de evaluación

La evaluación de la solución se estructuró en múltiples niveles, con el objetivo de validar tanto el funcionamiento técnico del sistema como la pertinencia y la calidad de las respuestas generadas por el agente conversacional. Las técnicas de validación empleadas fueron:

- **Pruebas funcionales por componente:** se verificó de forma aislada el correcto funcionamiento de cada módulo del sistema (autenticación, carga de archivos, búsqueda vectorial, generación de respuestas, sistema académico). Cada endpoint de la API fue probado verificando que las respuestas cumplieran con los formatos y los datos esperados.
- **Pruebas de integración del agente ReAct:** se evaluaron escenarios completos en los que el agente debía seleccionar herramientas, invocarlas y generar respuestas coherentes. Se diseñaron consultas representativas de los casos de uso principales:
 - Consultas sobre escolaridad (¿Qué materias estoy cursando?).
 - Consultas sobre las materias disponibles (¿En qué materias puedo inscribirme el próximo semestre?).
 - Consultas sobre planes de estudio (¿Cuál es el plan de estudios de la Licenciatura en Sistemas?).
 - Consultas basadas en documentos (¿Qué dice el reglamento sobre las correlativas?).
 - Consultas mixtas que requerían múltiples herramientas.
- **Pruebas de usuario aisladas:** se realizaron sesiones de uso con personas ajenas al equipo de desarrollo para evaluar la usabilidad de la interfaz conversacional y la pertinencia de las respuestas generadas. Se recopiló feedback cualitativo sobre la experiencia de uso, la claridad de las respuestas y las expectativas del usuario.

- **Pruebas de rendimiento:** se midió el tiempo de respuesta del sistema bajo condiciones normales de uso, evaluando la latencia de cada componente (búsqueda vectorial, invocación de herramientas, generación de respuesta por el LLM).
- **Validación de determinismo en herramientas académicas:** se verificó que las herramientas deterministas (como la validación de prerrequisitos y la consulta de escolaridad) devolvieran resultados correctos y consistentes, independientemente del modelo de lenguaje utilizado.

4.4.2 Resultados de las pruebas

A continuación se presentan los principales resultados obtenidos durante la evaluación del sistema:

Resultados de las pruebas funcionales.

- El sistema de autenticación (Auth0) funcionó correctamente en todos los escenarios probados, gestionando adecuadamente los tokens JWT y los roles de usuario.
- La carga e indexación de documentos se completaron exitosamente para los formatos soportados (PDF, Word, Excel, texto plano), generando embeddings en ChromaDB de forma consistente y respetando los permisos de visibilidad (público/privado).
- Los endpoints del sistema académico devolvieron datos correctos sobre escolaridad, materias disponibles y planes de estudio, validando correctamente los prerrequisitos en Python de forma determinista.

Resultados de las pruebas del agente.

- El agente ReAct seleccionó correctamente las herramientas en la mayoría de los casos de prueba. Para consultas directas (¿Qué materias estoy cursando?), el agente identificó de forma consistente la herramienta apropiada (get student schooling).
- En consultas complejas que requerían múltiples herramientas o razonamiento sobre los resultados, se observó que la calidad dependía significativamente del modelo de lenguaje empleado.

- Las respuestas basadas en documentos (vía search documents) resultaron precisas cuando los documentos relevantes estaban correctamente indexados en ChromaDB.
- El sistema de feedback permitió recopilar valoraciones de los usuarios, lo que evidenció una tendencia positiva en la satisfacción general con las respuestas del asistente.

Resultados de las pruebas con usuarios.

- Los usuarios valoraron positivamente la interfaz conversacional como canal de consulta, destacando su accesibilidad y la posibilidad de obtener respuestas inmediatas.
- Se identificó que los usuarios esperaban respuestas más rápidas en algunos casos, especialmente cuando el modelo de lenguaje requería más tiempo de procesamiento para las consultas complejas.
- La presentación de las fuentes utilizadas en cada respuesta fue valorada positivamente, ya que brinda transparencia sobre el origen de la información proporcionada.

Resultados de rendimiento.

- El tiempo de respuesta promedio para consultas simples se situó dentro de rangos aceptables, aunque dependió del hardware local disponible.
- Las consultas que requerían múltiples invocaciones de herramientas mostraron tiempos de respuesta mayores, proporcionales al número de herramientas invocadas.
- La búsqueda vectorial en ChromaDB resultó eficiente incluso con volúmenes moderados de documentos indexados.

5 Caso de aplicación

En esta sección se presenta un caso de aplicación de la interfaz conversacional para la planificación de estudios en un contexto académico específico. Se presenta una demostración del sistema en funcionamiento, recorriendo las principales funcionalidades mediante capturas de pantalla que ilustran la experiencia del usuario.

El objetivo de esta demostración es evidenciar el comportamiento del sistema ante consultas reales y mostrar cómo el agente conversacional interactúa con las herramientas disponibles para brindar respuestas basadas en datos académicos e institucionales.

5.1 Ventanas del sitio

En esta sección se presentan las principales vistas de la aplicación web, recorriendo el flujo completo de navegación desde el acceso al sistema hasta las funcionalidades disponibles para cada rol de usuario.

5.1.1 Página de bienvenida (Landing)

Al acceder a la aplicación, el usuario visualiza la página de bienvenida (Figura 5.1), que presenta el sistema como “Asistente de Planificación de Estudios” y resume sus principales características: subir y organizar documentos de estudio, hacer preguntas con ayuda de IA y obtener asistencia de aprendizaje personalizada. El botón “Ingresar” redirige al usuario al proceso de autenticación, lo que indica que el sistema está protegido por Auth0.



Figura 5.1: Página de bienvenida del sistema.

5.1.2 Inicio de sesión (Auth0)

Al presionar “Ingresar”, el navegador redirige a la pantalla de inicio de sesión de Auth0 (Figura 5.2), donde el usuario ingresa su correo electrónico y contraseña. Desde esta pantalla también es posible crear una cuenta nueva o recuperar la contraseña. Una vez autenticado con éxito, Auth0 devuelve un token *JWT* al *frontend* y redirige al usuario a la aplicación.

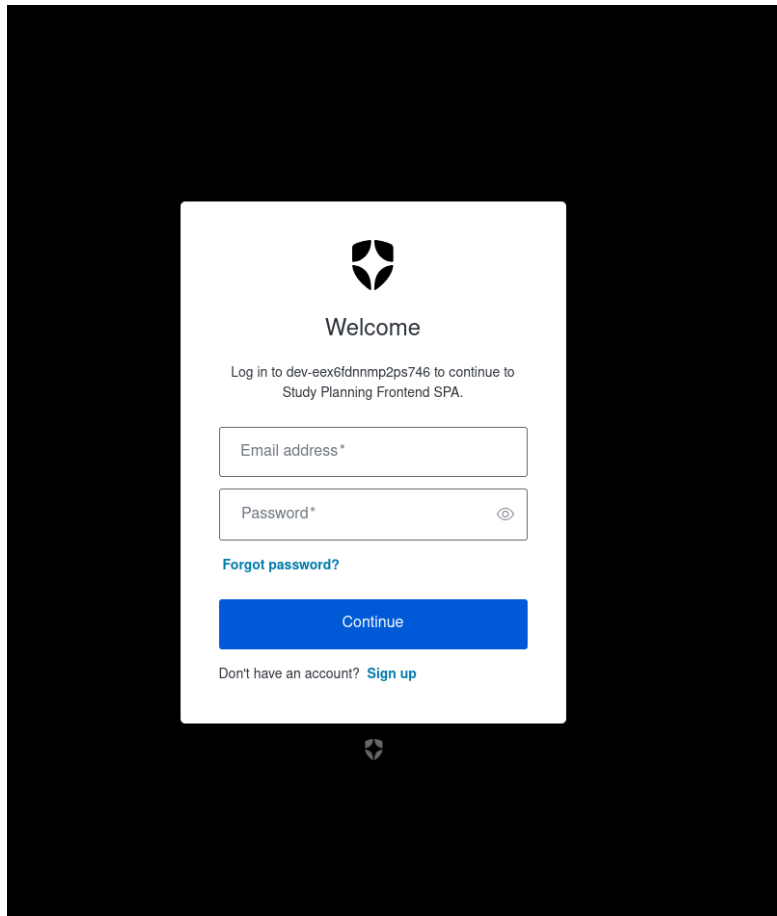


Figura 5.2: Pantalla de inicio de sesión con Auth0.

5.1.3 Perfil del estudiante

Tras iniciar sesión, el estudiante puede acceder a su perfil (Figura 5.3), que muestra su información personal (nombre, correo electrónico, rol y fecha de registro), estadísticas de uso (total de archivos subidos, consultas realizadas y almacenamiento utilizado), información de actividad (última actividad, fecha de creación de la cuenta) y los detalles técnicos de la cuenta (ID de usuario e ID de Auth0).



Figura 5.3: Vista del perfil del estudiante con estadísticas de uso.

5.1.4 Mis Cursos: progreso académico

La sección “Mis Cursos” (Figura 5.4) muestra el progreso académico del estudiante en su carrera (en este caso, *Computer Engineering*, con 320 créditos totales). En la pestaña “Mi Progreso” se visualiza un resumen con indicadores clave: materias completadas, materias actuales, créditos obtenidos y promedio. A continuación, se listan las materias en curso y las completadas, junto con sus calificaciones. Cada tarjeta de materia incluye el código, el nombre, los créditos y el estado, junto con íconos de acción para ver detalles, editar o eliminar.

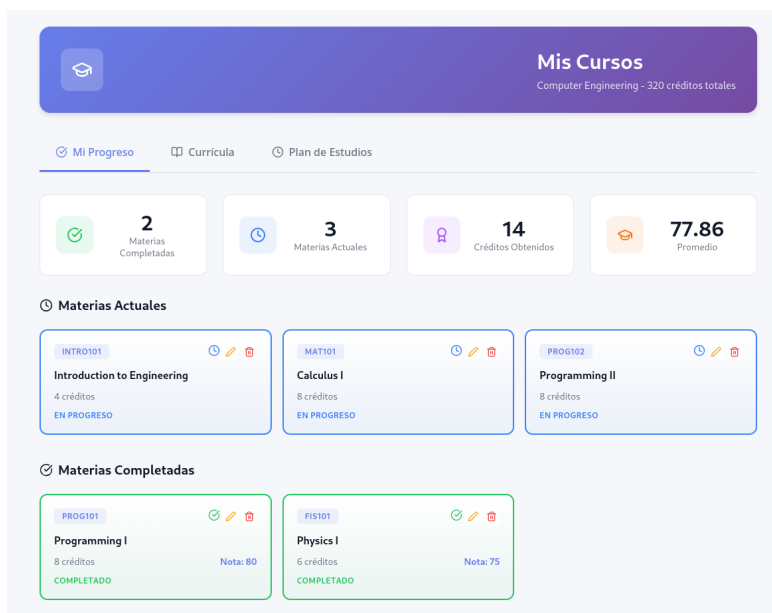


Figura 5.4: Vista “Mis Cursos” con el progreso académico del estudiante.

5.1.5 Currícula de la carrera

La pestaña “Currícula” (Figura 5.5) muestra la estructura completa de la carrera organizada por semestres. Cada materia se presenta en una tarjeta con un código de colores que indica su estado: verde con borde sólido para materias completadas (con nota visible), azul para materias en progreso, gris para materias bloqueadas (cuyos prerrequisitos aún no se han cumplido) y verde con borde punteado para materias disponibles para inscripción. También se muestran los prerrequisitos de cada materia, lo que permite al estudiante visualizar rápidamente su situación curricular.

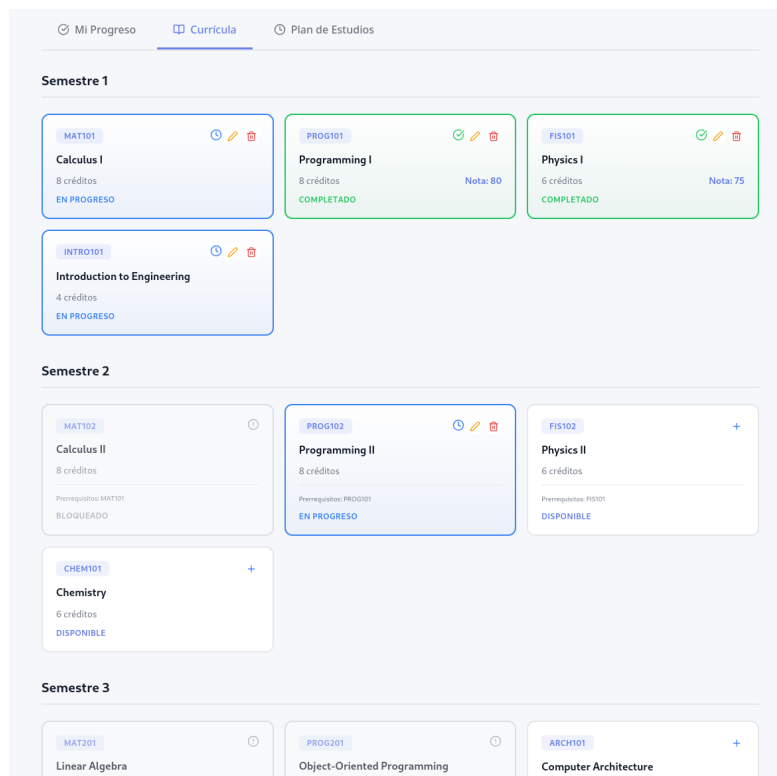


Figura 5.5: Vista de la currícula de la carrera organizada por semestres.

5.1.6 Plan de estudios

La pestaña “Plan de Estudios” (Figura 5.6) presenta las materias planificadas para el próximo período académico (por ejemplo, 2025-2). Se visualizan las materias disponibles con sus créditos y prerrequisitos, marcadas como “DISPONIBLE”, lo que indica que el estudiante cumple con los requisitos necesarios para inscribirse en ellas.

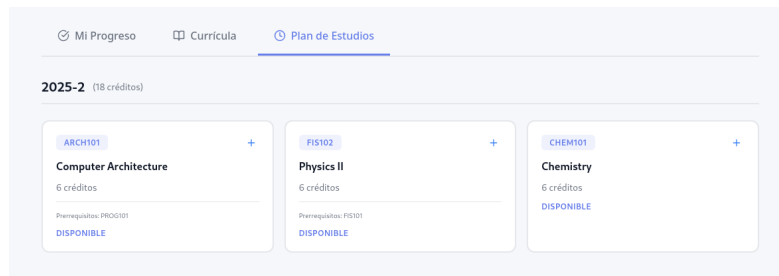


Figura 5.6: Vista del plan de estudios para el siguiente período académico.

5.1.7 Configuración del sistema

La vista de configuración (Figura 5.7) permite al usuario personalizar su experiencia. En la sección “Preferencias de Usuario” se puede seleccionar el idioma de la interfaz y de las respuestas del LLM, el número de fragmentos de contexto a recuperar para las consultas RAG y activar o desactivar la Generación Aumentada con Recuperación. En la sección “Modelo LLM Preferido” se listan los modelos disponibles en Ollama, lo que permite seleccionar el modelo a utilizar para las consultas.

 Configuración del Sistema

 Preferencias de Usuario

Idioma

Español

▼

Idioma para la interfaz y las respuestas del LLM.

Número de Fragmentos de Contexto

15 fragmentos

▼

Número de fragmentos de documentos a recuperar para contexto. Más fragmentos proporcionan más contexto pero pueden ser más lentos.

☒ Usar RAG (Generación Aumentada con Recuperación)

Cuando está activado, las consultas buscarán en tus documentos y usarán su contenido como contexto para las respuestas.

 Modelo LLM Preferido

Haz clic en un modelo abajo para seleccionarlo.

gemma3:4b

Size: 3338801804

Seleccionado

gemma3:latest

Size: 3338801804

llama3.1:8b

Size: 4920753328

qwen2.5:7b

Size: 4683087332

llama3.2:latest

Size: 2019393189

llama2:latest

Size: 3826793677

Figura 5.7: Vista de la configuración del sistema, con las preferencias y la selección del modelo LLM.

5.1.8 Estadísticas del sistema (administrador)

Los usuarios con rol de administrador acceden a un panel de estadísticas (Figura 5.8) que muestra información agregada sobre el uso del sistema. Se presentan indicadores globales (total de usuarios, total de consultas y almacenamiento), una tabla de usuarios registrados con su rol, la cantidad de archivos subidos, las consultas realizadas, el almacenamiento utilizado y la última actividad. Además, se muestran rankings de los archivos más valorados, tanto positivamente como negativamente, por los estudiantes, lo que proporciona información útil para evaluar la calidad de los documentos institucionales cargados en el sistema.

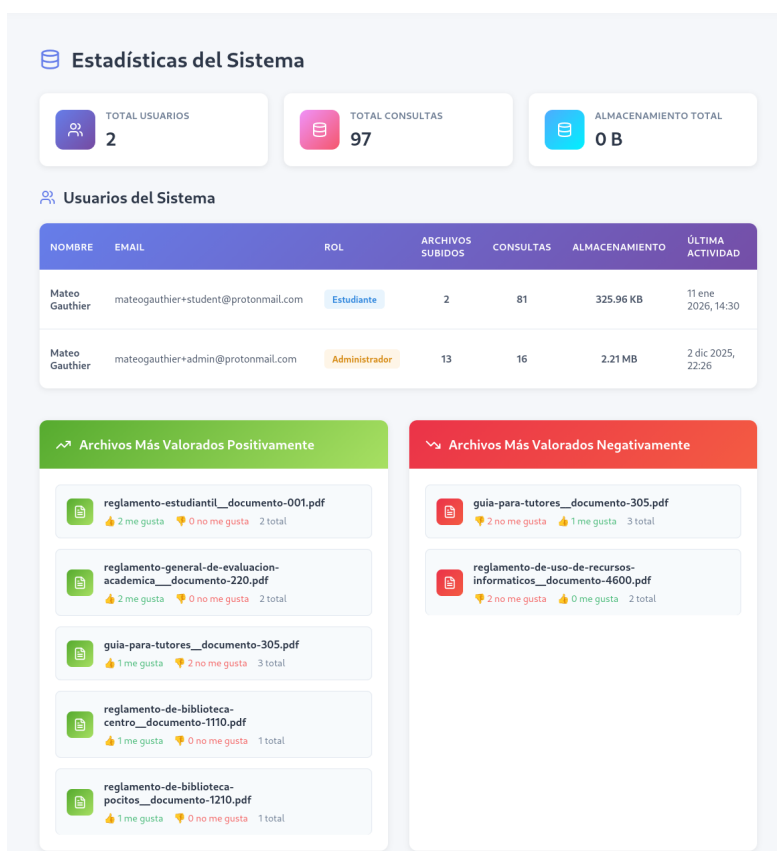


Figura 5.8: Panel de estadísticas del sistema visible para los administradores.

5.2 Consulta sobre materias aprobadas

En este escenario, el estudiante consulta al asistente: *¿Cuáles son las materias de las que ya tengo los créditos completos?* La Figura 5.9 muestra el estado académico del estudiante previo a la consulta, con 2 materias completadas (Programming I y Physics I), 3 en progreso y un promedio de 77.86.

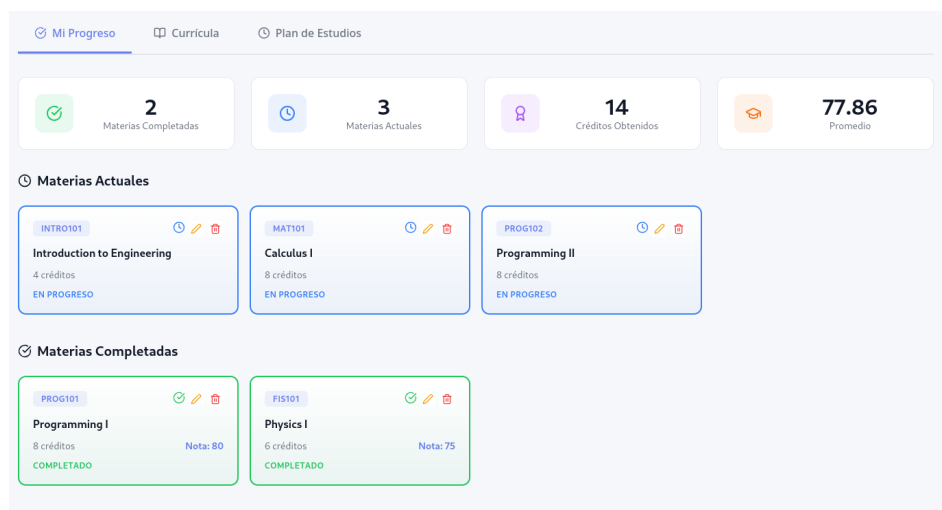


Figura 5.9: Progreso académico del estudiante previo a la consulta.

La Figura 5.10 muestra la respuesta del agente conversacional. Se observa el panel de “Agent Execution” con los 5 pasos realizados: el agente creó un plan estructurado, identificó la intención de la consulta, seleccionó la herramienta `get_completed_courses` y ejecutó la iteración correspondiente. El resultado confirma las materias completadas (Programación I – PROG101 y Física I – FIS101), los 14 créditos obtenidos y el promedio de 77.86 %, lo cual coincide con los datos reales del sistema académico. Al final de la respuesta, el asistente ofrece proactivamente continuar con la planificación del siguiente semestre.

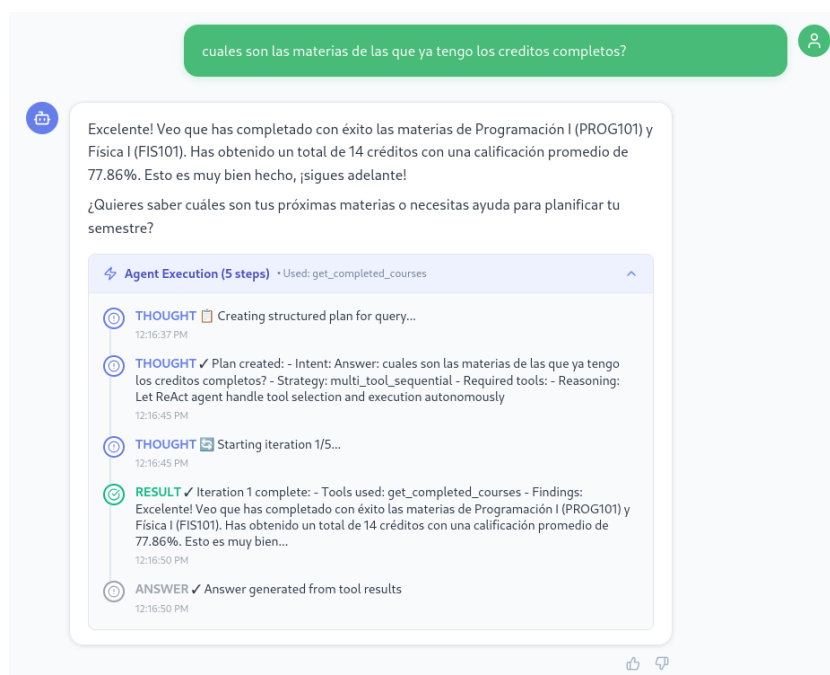


Figura 5.10: Respuesta del agente que muestra las materias completadas y los pasos de ejecución del agente ReAct.

5.3 Consulta de materias disponibles para inscripción

En este escenario, el estudiante inicia una conversación saludando y luego pregunta: *¿Puedes decirme a qué materias me puedo inscribir?* La Figura 5.11 muestra la interacción completa: el agente responde con un listado de tres materias disponibles para la inscripción, detallando para cada una su código, nombre y justificación. Las materias identificadas son Física II (FIS102), Química (CHEM101) y Arquitectura de Computadoras (ARCH101), lo cual coincide con las materias marcadas como “DISPONIBLE” en la vista de currícula (Figura 5.5). El agente indica los prerrequisitos cumplidos en cada caso y sugiere verificar los horarios antes de inscribirse.

La Figura 5.12 muestra los pasos internos del agente ReAct. Se observa que el agente utilizó la herramienta `get_available_courses` en 5 pasos: creó un plan estructurado, identificó la intención de la consulta, ejecutó la herramienta y generó

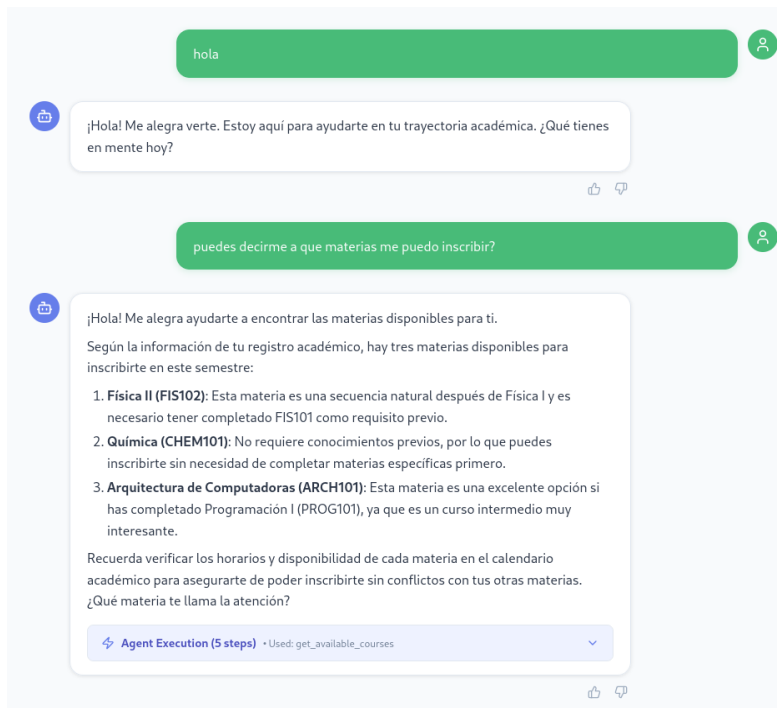


Figura 5.11: Respuesta del agente con las materias disponibles para la inscripción y sus respectivas justificaciones.

la respuesta a partir de los resultados. La validación de prerrequisitos se realizó de forma determinista en Python, garantizando que las materias sugeridas sean efectivamente inscribibles.

¡Hola! Me alegra ayudarte a encontrar las materias disponibles para ti.

Según la información de tu registro académico, hay tres materias disponibles para inscribirte en este semestre:

1. **Física II (FIS102)**: Esta materia es una secuencia natural después de Física I y es necesario tener completado FIS101 como requisito previo.
2. **Química (CHEM101)**: No requiere conocimientos previos, por lo que puedes inscribirte sin necesidad de completar materias específicas primero.
3. **Arquitectura de Computadoras (ARCH101)**: Esta materia es una excelente opción si has completado Programación I (PROG101), ya que es un curso intermedio muy interesante.

Recuerda verificar los horarios y disponibilidad de cada materia en el calendario académico para asegurarte de poder inscribirte sin conflictos con tus otras materias. ¿Qué materia te llama la atención?

🔗 Agent Execution (5 steps) • Used: get_available_courses

- 🕒 12:13:21 PM
THOUGHT 📝 Creating structured plan for query...
- 🕒 12:13:40 PM
THOUGHT ✓ Plan created: - Intent: Answer: puedes decirme a que materias me puedo inscribir? - Strategy: multi_tool_sequential - Required tools: - Reasoning: Let ReAct agent handle tool selection and execution autonomously
- 🕒 12:13:40 PM
THOUGHT 🔄 Starting iteration 1/5...
- 🕒 12:13:48 PM
RESULT ✓ Iteration 1 complete: - Tools used: get_available_courses - Findings: ¡Hola! Me alegra ayudarte a encontrar las materias disponibles para ti. Según la información de tu registro académico, hay tres materias disponibles para inscribirte en este semestre: 1. **Física II...**
- 🕒 12:13:48 PM
ANSWER ✓ Answer generated from tool results

Figura 5.12: Detalle de los pasos de ejecución del agente ReAct para la consulta de materias disponibles.

5.4 Manejo de archivos

La Figura 5.13 muestra la vista de archivos públicos, cargados por el administrador. Se observa la zona de carga mediante *drag-and-drop* (soportando archivos PDF, Word, Excel, texto y Markdown) y el listado de documentos institucionales disponibles (reglamentos, normas, códigos), cada uno con su tamaño, número de *chunks* generados, fecha de subida y estadísticas de uso.

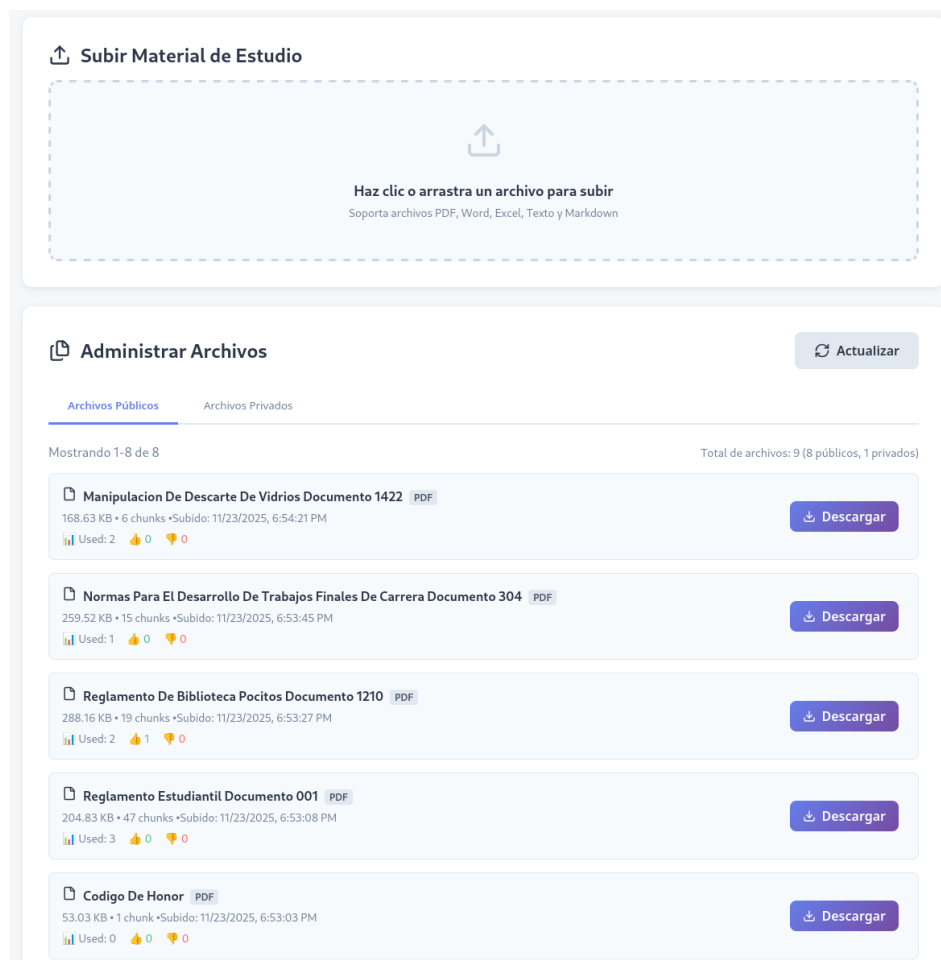


Figura 5.13: Vista de los archivos públicos con la zona de carga y el listado de documentos institucionales.

La Figura 5.14 muestra la pestaña de archivos privados del estudiante. En este caso, el estudiante subió su escolaridad en formato PDF, que fue fragmentada

en 8 *chunks* e indexada en ChromaDB. El sistema implementa *multi-tenancy*: los archivos privados solo son visibles para su propietario, mientras que los públicos están disponibles para todos los usuarios.

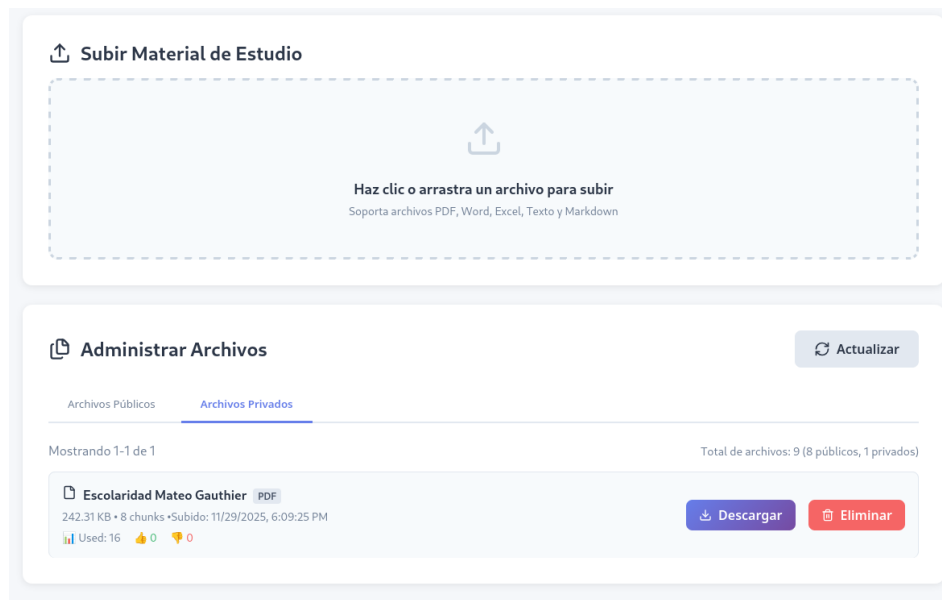


Figura 5.14: Vista de los archivos privados del estudiante.

5.5 Feedback de estudiantes

La Figura 5.15 muestra cómo el estudiante envía feedback sobre una respuesta del agente. Debajo de cada respuesta, el usuario puede valorarla positiva o negativamente y agregar un comentario opcional (en este caso, “Muestra las materias de manera correcta”).

La Figura 5.16 muestra el panel de administración en el que se visualizan los comentarios recopilados (6 en total: 4 positivos y 2 negativos). Cada comentario incluye el usuario, la fecha, el texto, la valoración (*like/dislike*) y los archivos referenciados en la respuesta evaluada.

La Figura 5.17 muestra la funcionalidad de “Generar Resumen IA”, que produce automáticamente un análisis del feedback mediante el modelo de lenguaje. El resumen identifica el sentimiento general (positivo), los temas principales y los elogios comunes de los estudiantes, proporcionando a los administradores una visión agregada de la satisfacción con el sistema.

cuales son las materias de las que ya tengo los creditos completos?

Excelente! Veo que has completado con éxito las materias de Programación I (PROG101) y Física I (FIS101). Has obtenido un total de 14 créditos con una calificación promedio de 77.86%. Esto es muy bien hecho, ¡sigues adelante!

¿Quieres saber cuáles son tus próximas materias o necesitas ayuda para planificar tu semestre?

Agent Execution (5 steps) • Used: get_completed_courses

Add a comment (optional)

Muestra las materias de manera correcta

Submit Feedback Cancel

Figura 5.15: Envío de feedback por parte del estudiante sobre una respuesta del agente.

Panel de Comentarios de Estudiantes

Ver y analizar comentarios de estudiantes para mejorar el sistema

Total de Comentarios 6	Comentarios Positivos 4	Comentarios Negativos 2	Con Comentarios 6
---------------------------	----------------------------	----------------------------	----------------------

Mostrar Filtros Generar Resumen IA

Comentarios (6)

mateogauthier+student@protonmail.com
2/8/2026, 12:20:17 PM
LIKE

"Muestra las materias de manera correcta"

ID de Mensaje: 69887eb2... Conversación: 69887db1...

mateogauthier+admin@protonmail.com
11/23/2025, 6:55:38 PM
LIKE

"Me gusta que se muestren alternativas a la pregunta"

Archivos Referenciados:
reglamento-de-biblioteca-centro_documento-1110.pdf reglamento-de-biblioteca-pocitos_documento-1210.pdf

ID de Mensaje: 6923587a... Conversación: 69235872...

mateogauthier+student@protonmail.com
11/17/2025, 11:36:08 AM
LIKE

"Muy buen facil acceso a las fuentes de documentos"

Archivos Referenciados:
reglamento-estudianti_documento-001.pdf reglamento-general-de-evaluacion-academica_documento-220.pdf

ID de Mensaje: 691b071c... Conversación: 691a4c9...

mateogauthier+admin@protonmail.com
11/16/2025, 10:23:50 PM
DISLIKE

"Me gustaria que las listas se vieran de manera mas prolija"

Archivos Referenciados:
reglamento-de-uso-de-recursos-informaticos_documento-4600.pdf guia-para-tutores_documento-305.pdf

ID de Mensaje: 69191380... Conversación: 69191177...

Figura 5.16: Panel de comentarios de estudiantes visible para los administradores.

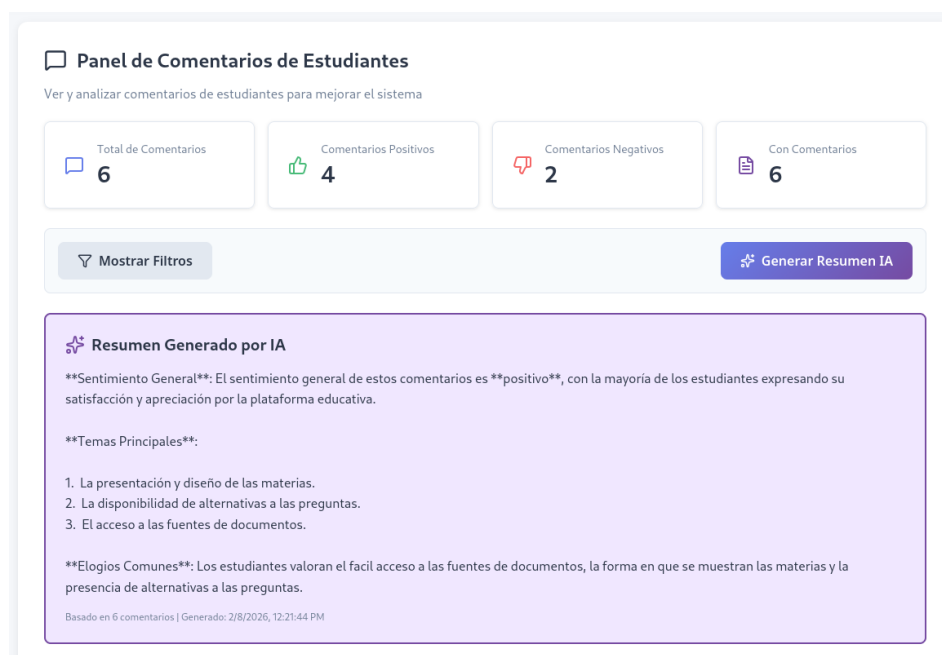


Figura 5.17: Resumen generado por IA del feedback de los estudiantes.

5.6 Recomendación de materias a cursar

En este escenario, el estudiante consulta al asistente: “¿En qué materias me recomiendas inscribirme el próximo semestre?” La herramienta `get_course_recommendations` genera sugerencias personalizadas mediante distintos algoritmos de Machine Learning, mientras que `get_completed_courses` aporta contexto sobre el desempeño académico del estudiante. Además, el agente puede utilizar `create_study_plan` para proponer un plan de estudios para el próximo semestre.

En el panel de Agent Execution se observa cómo el agente interpreta la consulta, selecciona las herramientas necesarias y construye la respuesta. El resultado incluye las materias recomendadas con sus justificaciones, un plan sugerido y una indicación para ajustarlo según las preferencias del estudiante. Las recomendaciones se basan en los cursos ya completados y en el rendimiento académico previo.

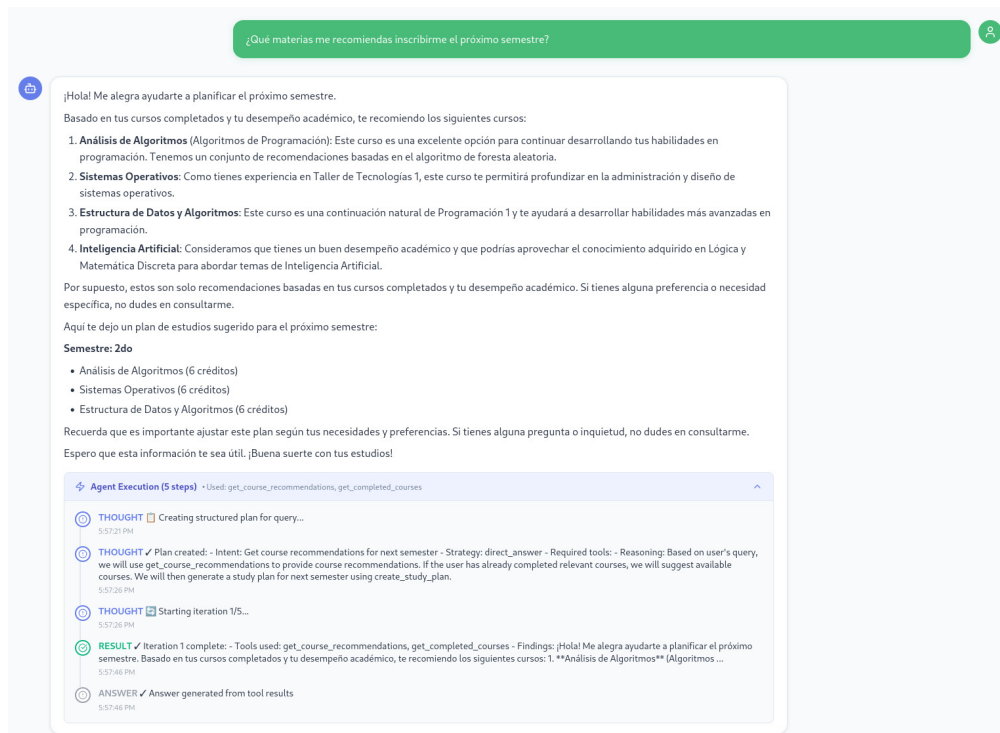


Figura 5.18: Respuesta del agente con recomendaciones de materias y el plan de estudios sugerido.

6 Discusión y Limitaciones

En este capítulo se presentan las discusiones derivadas del caso de aplicación, las limitaciones técnicas identificadas durante el desarrollo y la evaluación del sistema, así como los aspectos éticos de la investigación. El objetivo es consolidar un análisis crítico integral que permita contextualizar los resultados obtenidos y orientar las futuras iteraciones del proyecto.

6.1 Discusión del caso de aplicación

El caso de aplicación presentado en el capítulo anterior permitió validar el funcionamiento del sistema en un escenario académico concreto. A partir de las pruebas realizadas, se identificaron tanto fortalezas como áreas de mejora en la solución propuesta.

Entre las fortalezas observadas se destaca la capacidad del agente conversacional para interpretar consultas en lenguaje natural y utilizar de forma autónoma las herramientas disponibles, recuperando información académica precisa sobre materias completadas, materias disponibles para inscripción y currícula de la carrera. La arquitectura RAG demostró ser efectiva para fundamentar las respuestas en documentos institucionales verificables, lo que redujo la incidencia de alucinaciones en comparación con un LLM sin contexto externo.

El sistema de feedback integrado permitió recopilar valoraciones de los usuarios, lo que evidenció una recepción mayoritariamente positiva. La funcionalidad de generar resúmenes automáticos del feedback mediante IA proporcionó a los administradores una herramienta útil para evaluar la calidad del servicio sin necesidad de revisar cada comentario individualmente.

Sin embargo, también se identificaron limitaciones relevantes. En conversaciones prolongadas o con consultas de alta complejidad, la calidad de las respuestas

del modelo local tendió a deteriorarse, mostrando una mayor propensión a las alucinaciones y a la pérdida de coherencia. Además, la dependencia de hardware local con capacidad de cómputo suficiente supuso una barrera para el despliegue en entornos institucionales con recursos limitados.

La validación determinista de prerequisites en Python resultó ser una decisión acertada, ya que garantizó que las materias sugeridas como disponibles fueran efectivamente inscribibles, sin depender de la precisión del modelo de lenguaje para esta tarea crítica.

Estas observaciones refuerzan la viabilidad de la solución como prueba de concepto, al tiempo que señalan las áreas donde se requiere mayor maduración para una eventual implementación en producción.

6.2 Limitaciones técnicas

Durante la evaluación y el desarrollo del sistema se identificaron las siguientes limitaciones técnicas:

6.2.1 Limitaciones de recursos y costos

El hosting en servicios cloud como AWS o Azure requiere recursos computacionales significativos: abundante memoria RAM, procesadores potentes y acceso a GPUs para ejecutar los LLMs de manera eficiente. Los modelos grandes requieren aproximadamente 30 GB de memoria, lo que resulta prohibitivo para servidores de bajo costo o instancias gratuitas.

Durante las pruebas de despliegue en AWS Free Tier (2 GB de RAM, 2 cores), el sistema resultó completamente inutilizable: la página de inicio de sesión tardaba más de un minuto en cargar y el servidor se bloqueaba por falta de recursos al ejecutar los contenedores Docker. Incluso la conexión SSH se volvía lenta debido a la sobrecarga del sistema.

Esta limitación de recursos es una de las principales razones por las que el sistema se ejecuta localmente mediante Docker Compose, utilizando Ollama para ejecutar modelos de código abierto que requieren menos recursos que los modelos comerciales.

6.2.2 Limitaciones de los modelos locales

Los LLMs ejecutados localmente presentan limitaciones inherentes que no pueden superarse sin hardware especializado. A medida que la complejidad de la tarea aumenta y se acumulan mensajes en la conversación, los modelos locales comienzan a mostrar comportamientos problemáticos:

- **Alucinaciones:** el modelo genera información falsa o inventa materias que no existen.
- **Degradación de la calidad:** las respuestas se vuelven menos coherentes.
- **Límites de complejidad:** existe un punto en el que el modelo simplemente no puede procesar tareas más complejas de manera confiable.

Estas limitaciones son inherentes a los modelos locales disponibles y no pueden resolverse sin migrar a modelos más grandes ejecutados en infraestructura especializada (servidores con GPUs dedicadas), lo que implica costos prohibitivos para una prueba de concepto.

6.2.3 Limitaciones del agente en acciones

Para evitar alucinaciones y errores del LLM, el sistema limita las herramientas disponibles al agente a aquellas que no requieren parámetros proporcionados por el usuario. Por ejemplo:

- `get_completed_courses()`, `get_course_recommendations()`: no requieren parámetros; el sistema identifica al estudiante autenticado y obtiene sus datos.
- `get_available_courses()`: no requiere parámetros; el sistema determina automáticamente qué materias están disponibles.

Esta limitación evita que el LLM invente parámetros o utilice datos incorrectos, pero restringe significativamente las acciones que el agente puede realizar. Por ejemplo, el agente no puede inscribir a un estudiante en una materia porque eso requeriría que el LLM proporcionara el identificador de la materia, lo cual podría derivar en una inscripción incorrecta si el modelo alucina.

Como resultado, el agente se limita principalmente a:

- Recopilar información de endpoints predefinidos.
- Generar recomendaciones basadas en datos recuperados.
- Presentar información en formato de texto, tablas o Markdown.

Las acciones que requieren parámetros del usuario (como inscribirse en una materia) quedan fuera del alcance del sistema actual, ya que no se puede garantizar el determinismo necesario para operaciones críticas del sistema académico.

6.3 Aspectos éticos de la investigación

En esta sección se discuten las implicancias éticas y las consideraciones críticas derivadas del desarrollo y de la potencial implementación de la interfaz conversacional para la planificación de estudios. La integración de la inteligencia artificial (IA) en procesos académicos conlleva responsabilidades en la gestión de la información, la confiabilidad de las respuestas y el impacto de las recomendaciones en la trayectoria de los estudiantes. Además, se analizan las restricciones impuestas por los recursos computacionales disponibles y la sostenibilidad a largo plazo de las soluciones basadas en LLMs.

6.3.1 Privacidad y protección de datos institucionales y personales

La privacidad constituye un pilar ético fundamental. Dado que el sistema procesa información sensible, como la escolaridad y el desempeño académico, existe un riesgo inherente al uso de LLMs en la nube.

Para mitigar esto, el proyecto propone la ejecución local de modelos mediante Ollama. Esta decisión asegura que los datos académicos no se transfieran a servidores de terceros ni se utilicen para entrenar modelos externos. El almacenamiento se realiza en infraestructura controlada, lo que garantiza que la información sensible permanezca bajo el control de la institución.

No obstante, cabe señalar que la ejecución local no es la única alternativa para preservar la privacidad. Actualmente, diversos proveedores de LLMs comerciales ofrecen planes empresariales con garantías contractuales de que los datos enviados no se utilizarán para entrenar sus modelos y de que la comunicación se realiza a través de canales cifrados y autenticados. En el contexto de este proyecto, donde

la aplicación web ya implementa la autenticación mediante Auth0 y cada usuario solo accede a su propia información, la integración con un LLM en la nube, bajo estas condiciones, representaría una alternativa viable que permitiría acceder a modelos más potentes sin comprometer la privacidad de los datos institucionales. Esta posibilidad se considera una línea de trabajo futura que permitiría evaluar la mejora de la calidad de las respuestas mediante modelos de mayor capacidad, manteniendo las garantías de protección de datos requeridas por la institución.

6.3.2 Riesgos de alucinaciones y legitimidad de la información

Un desafío ético recurrente en los LLMs es la generación de respuestas plausibles pero incorrectas, conocidas como alucinaciones. En un contexto de planificación de estudios, una recomendación errónea sobre una materia o un requisito de créditos podría perjudicar seriamente el progreso curricular del estudiante. Los LLM, por sí mismos, suelen presentar dificultades para resolver problemas científicos o técnicos complejos que requieren resultados intermedios de múltiples pasos [39].

La legitimidad de la información se aborda mediante la arquitectura RAG (Retrieval-Augmented Generation). Este enfoque busca que el modelo fundamente sus respuestas en documentos institucionales verificables (planes de estudio y reglamentos) en lugar de depender de su conocimiento interno estático. Sin embargo, para que esta arquitectura sea efectiva, el agente debe poseer una alta capacidad de tool calling (llamadas a herramientas). Investigaciones recientes demuestran que el uso de herramientas externas permite a los LLM transformarse en agentes poderosos capaces de realizar tareas complejas mediante razonamiento dinámico [39].

A pesar de estas ventajas, existe un riesgo crítico asociado a la eficacia de los modelos en la ejecución de dichas herramientas. Los modelos accesibles para ejecución local o en versiones reducidas muestran una baja efectividad en benchmarks de uso de herramientas para agentes. Por ejemplo, en el benchmark TM-BENCH, se ha observado que los modelos con capacidades de razonamiento más limitadas (como o3-mini) presentan un desempeño significativamente inferior en comparación con modelos de última generación, lo que aumenta el riesgo de errores en el flujo de trabajo del agente [39]. Esto implica que, en un sistema local, la incapacidad del modelo para invocar correctamente las herramientas de búsqueda o de validación académica puede derivar en respuestas sin fundamento real y comprometer la integridad de la asistencia proporcionada.

Es éticamente necesario que el sistema no solo informe al usuario de que las respuestas son generadas por IA, sino que también implemente mecanismos de autocorrección en bucle cerrado (closed-loop self-correction) para diagnosticar errores de ejecución y minimizar la entrega de información ilegítima [39].

6.3.3 Sostenibilidad e incertidumbre de la industria de los LLMs

La investigación reconoce la preocupación por la sostenibilidad de la industria de la IA generativa. Actualmente, el sector depende de una inversión masiva en una tecnología que aún no ha demostrado ser plenamente rentable de forma independiente.

Depender de proveedores comerciales conlleva riesgos de continuidad ante cambios en los modelos de suscripción o posibles cierres de empresas. El uso de modelos de código abierto en este proyecto busca reducir esta vulnerabilidad, apostando por soluciones que puedan ser mantenidas de forma autónoma por la facultad, sin depender exclusivamente de un mercado tecnológico incierto.

6.3.4 Análisis de costos y viabilidad económica del hosting

El despliegue de sistemas basados en IA implica un costo de infraestructura significativamente mayor que el de los sistemas de gestión tradicionales. La ejecución eficiente de LLMs requiere recursos computacionales avanzados, incluyendo abundante memoria RAM y unidades de procesamiento gráfico (GPU). Investigaciones recientes sobre flujos de trabajo científicos automatizados refuerzan la premisa de que la capacidad computacional, y no el recurso humano, es actualmente el factor limitante en el ritmo de descubrimiento y de procesamiento de la información académica.

Se han identificado limitaciones críticas y datos comparativos sobre la viabilidad económica:

- **Intensidad de recursos por tarea:** Los modelos capaces de razonamiento complejo requieren aproximadamente 30 GB de memoria para operar con fluidez. En entornos de agentes dinámicos como TOOLMAKER, se ha observado que la ejecución de tareas multietapa requiere, en promedio, 21,8

acciones por cada herramienta creada. Esta alta tasa de interacción con el sistema operativo y con las bases de datos externas implica un consumo constante de ciclos de CPU y de memoria que eleva drásticamente los costos operativos frente a una aplicación convencional.

- **Costo operativo por consulta:** Si bien el presente proyecto propone una ejecución local para preservar la privacidad, el análisis de benchmarks comerciales evidencia la magnitud del gasto. Por ejemplo, el costo promedio por implementación de una tarea científica compleja utilizando modelos de última generación (como GPT-4o) se estima en \$0.94 USD por operación, llegando en casos complejos hasta los \$2.90 USD [39].
- **Degradación por reducción de costos:** Existe una correlación directa entre la reducción de los costos de infraestructura y la pérdida de eficacia. Al utilizar modelos más económicos o versiones reducidas (como o3-mini), los costos por operación pueden bajar a un promedio de \$0.55 USD, pero esto conlleva una degradación significativa en la precisión y capacidad de resolución de problemas complejos [39].

Éticamente, la institución debe evaluar si el beneficio de una interfaz conversacional justifica un gasto operativo que puede ser varias veces superior al de las soluciones tradicionales. La sostenibilidad económica a largo plazo depende de encontrar un equilibrio entre el uso de técnicas como la síntesis de información (que puede reducir el número de acciones y los costos hasta en un 25 %) y el mantenimiento de hardware especializado propio que amortice los costos de las APIs comerciales.

7 Conclusiones

El presente trabajo tuvo como objetivo diseñar y desarrollar una prueba de concepto de una interfaz conversacional para la planificación de estudios en la Universidad ORT Uruguay. Para ello, se implementó una arquitectura basada en un agente inteligente de React que integra modelos de lenguaje de código abierto, ejecutados localmente mediante Ollama, con una base de conocimiento institucional gestionada mediante ChromaDB y orquestada por LangGraph.

7.1 Principales resultados

La prueba de concepto demostró la viabilidad de emplear una arquitectura de agentes junto con herramientas especializadas para asistir a los estudiantes en su planificación académica. Entre los principales resultados se destacan:

El agente conversacional logró interpretar consultas en lenguaje natural y seleccionar de forma autónoma las herramientas adecuadas para responderlas, abarcando escenarios como la consulta de materias aprobadas, la identificación de materias disponibles para la inscripción y la generación de recomendaciones personalizadas basadas en algoritmos de Machine Learning desarrollados en el proyecto Study Buddy [13].

La evolución de una arquitectura RAG tradicional hacia un sistema de agentes ReAct constituyó una decisión arquitectónica clave. Esta transición permitió unificar el razonamiento y la recuperación de información en un único flujo, resolviendo el conflicto técnico que impedía el funcionamiento simultáneo de ambos enfoques. El agente asumió las responsabilidades del RAG mediante herramientas específicas, lo que le otorgó mayor control sobre cuándo y cómo acceder a los documentos.

La implementación de herramientas deterministas en Python para la validación

de prerequisites y la consulta de la escolaridad resultó fundamental para garantizar la confiabilidad de las recomendaciones académicas, evitando delegar en el modelo de lenguaje cálculos lógicos críticos.

El sistema de feedback integrado, junto con la funcionalidad de generación de resúmenes automáticos mediante IA, proporcionó a los administradores una herramienta útil para evaluar la calidad del servicio de forma agregada.

7.2 Aplicación de los resultados

La solución desarrollada permite que un estudiante se autentique de forma segura, cargue documentos relevantes, formule preguntas en lenguaje natural y reciba respuestas basadas en información institucional verificable. El sistema cumple con los requisitos funcionales y no funcionales definidos en la Sección 3.2, organizados en una arquitectura de tres capas (Backend, Frontend e Infraestructura) desplegable mediante un único comando de Docker Compose.

La validación con usuarios ajenos al equipo de desarrollo evidenció una recepción positiva de la interfaz conversacional como canal de consulta, destacándose la accesibilidad, la inmediatez de las respuestas y la transparencia en la presentación de las fuentes utilizadas.

7.3 Limitaciones

El desarrollo y la evaluación del sistema revelaron un conjunto de tensiones inherentes a las soluciones de IA en contextos académicos. La privacidad de los datos motivó la ejecución local, pero esta limitó la calidad de los modelos disponibles. La reducción de alucinaciones requirió mecanismos de verificación (RAG, herramientas deterministas), cuya eficacia depende de la capacidad del modelo para invocar correctamente dichas herramientas. La sostenibilidad económica del hosting planteó desafíos que trascienden las decisiones técnicas e involucran definiciones institucionales sobre la inversión en infraestructura tecnológica.

En particular, los modelos locales mostraron una tendencia a la degradación en conversaciones prolongadas o ante consultas de alta complejidad. Asimismo, la dependencia de hardware con capacidad de cómputo suficiente supuso una barrera para el despliegue en entornos con recursos limitados. Estas limitaciones, junto con

la restricción de las herramientas del agente a operaciones sin parámetros proporcionados por el LLM, delimitaron el alcance funcional del sistema a operaciones de consulta y recomendación, excluyendo acciones transaccionales como la inscripción directa en materias.

7.4 Líneas de trabajo futuro

A partir de los resultados y las limitaciones identificadas, se proponen las siguientes líneas de trabajo futuro:

La integración de LLMs comerciales en la nube, bajo planes empresariales que garanticen la no utilización de datos para el entrenamiento, constituye una alternativa viable para mejorar la calidad de las respuestas sin comprometer la privacidad. Dado que la aplicación ya implementa la autenticación mediante Auth0 y el aislamiento de datos por usuario, esta migración resultaría técnicamente factible y permitiría evaluar el impacto de modelos más potentes en la precisión del agente.

La ampliación de las herramientas del agente para soportar acciones transaccionales mediante mecanismos de confirmación explícita por parte del usuario incrementaría significativamente la utilidad del sistema.

La implementación de mecanismos de autocorrección en bucle cerrado para diagnosticar errores de ejecución del agente reduciría la entrega de información incorrecta y fortalecería la confiabilidad del sistema.

La validación a mayor escala, con una población estudiantil más amplia y de múltiples carreras, permitiría evaluar la generalización de la solución y su adaptabilidad a distintos contextos curriculares.

Finalmente, la exploración de técnicas de optimización, como la síntesis de información y la reducción de las acciones del agente, podría mejorar tanto el rendimiento como la sostenibilidad económica del sistema en un eventual despliegue institucional.

8 Bibliografía

- [1] A. I. Oviedo Carrascal and J. Jiménez Giraldo, “Minería de datos educativos: Análisis del desempeño de estudiantes de ingeniería en las pruebas saber-pro,” *Revista Politécnica*, vol. 15, no. 29, pp. 128–140, 2019.
- [2] T. Sakirin and R. Ben Said, “User preferences for chatgpt-powered conversational interfaces versus traditional methods,” *Mesopotamian journal of Computer Science*, vol. 2023, pp. 22–28, 2023.
- [3] J. M. Helm, A. M. Swiergosz, H. S. Haeberle, J. M. Karnuta, J. L. Schaffer, V. E. Krebs, A. I. Spitzer, and P. N. Ramkumar, “Machine learning and artificial intelligence: definitions, applications, and future directions,” *Current reviews in musculoskeletal medicine*, vol. 13, no. 1, pp. 69–76, 2020.
- [4] A. Salemi and H. Zamani, “Evaluating retrieval quality in retrieval-augmented generation,” in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR ’24. New York, NY, USA: Association for Computing Machinery, 2024.
- [5] N. Duarte, M. Gauthier, and F. Rodríguez, “Interfaz conversacional para la planificación de estudios,” 2026. [Online]. Available: <https://github.com/mateogauthier/Conversational-interface-for-study-planning>
- [6] M. Giletta, A. Giordano, N. Mercaú, P. Orden, and V. Villarreal, “Inteligencia artificial: definiciones en disputa,” *Sociales Investiga*, no. 9, pp. 20–33, 2020.
- [7] J. M. Helm, A. M. Swiergosz, H. S. Haeberle, J. M. Karnuta, J. L. Schaffer, V. E. Krebs, A. I. Spitzer, and P. N. Ramkumar, “Machine learning and artificial intelligence: definitions, applications, and future directions,” *Current reviews in musculoskeletal medicine*, vol. 13, no. 1, pp. 69–76, 2020.
- [8] Z. M. Altukhi and S. Pradhan, “Systematic literature review: Explainable ai definitions and challenges in education,” *arXiv preprint arXiv:2504.02910*, 2025.

- [9] C. Romero and S. Ventura, “Educational data mining and learning analytics: An updated survey,” *WIREs Data Mining and Knowledge Discovery*, vol. 10, no. 3, p. e1355, 2020.
- [10] G. Casalino, G. Castellano, and C. Mencar, “Incremental and adaptive fuzzy clustering for virtual learning environments data analysis,” in *2019 International Conference on Virtual Reality and Visualization (ICVRV)*, 2019.
- [11] W. M. P. van der Aalst, *Process Mining - Data Science in Action, Second Edition*. Springer, 2016.
- [12] D. Calegari and A. Delgado, “A systematic review on process mining for curricular analysis,” *WIREs Data Mining and Knowledge Discovery*, vol. 15, no. 2, p. e70005, 2025.
- [13] A. Fernández and S. Cozzano, “Ia aplicada a la planificación de estudios,” Tesis de grado Licenciatura en Sistemas, Universidad ORT Uruguay, 2025.
- [14] I. Reis, D. Baron, and S. Shahaf, “Probabilistic random forest: A machine learning algorithm for noisy data sets,” *The Astronomical Journal*, vol. 157, no. 1, p. 16, 2019.
- [15] V. Xiang, C. Snell, K. Gandhi, A. Albalak, A. Singh, C. Blagden, D. Phung, R. Rafailov, N. Lile, D. Mahan, L. Castricato, J.-P. Fränken, N. Haber, and C. Finn, “Towards system 2 reasoning in llms: Learning how to think with meta chain-of-thought,” 2025.
- [16] J. Lanchantin, A. Chen, J. Lan, X. Li, S. Saha, T. Wang, J. Xu, P. Yu, W. Yuan, J. Weston, S. Sukhbaatar, and I. Kulikov, “Bridging offline and online reinforcement learning for llms,” 2025.
- [17] S. Wang, J. Asilis, Ömer Faruk Akgül, E. B. Bilgin, O. Liu, and W. Neiswanger, “Tina: Tiny reasoning models via lora,” 2025.
- [18] D. Li, S. Cao, T. Griggs, K. Hakhamaneshi, S. G. Patil, S. Liu, X. Mo, E. Tang, S. Hegde, M. Zaharia, J. E. Gonzalez, and I. Stoica, “Llms can easily learn to reason from demonstrations: Structure, not content, is what matters!” 2025.
- [19] A. Kumar T, “Evaluation of fine-tuned llm using monsterapi,” https://medium.com/@avikumart_/evaluation-of-fine-tuned-llm-using-monsterapi-a67a7714a65b, December 2023, accessed: 2025-10-19.

- [20] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, The Llama Team, and AI @ Meta, “The llama 3 herd of models,” 2024.
- [21] T. Gao, J. Jin, Z. T. Ke, and G. Moryoussef, “A comparison of deepseek and other llms,” *arXiv preprint arXiv:2502.03688*, Feb 2025.
- [22] K. Kopanov and T. Atanasova, “A comparative pattern analysis of qwen 2.5 and gemma 3 text generation,” *WSEAS TRANSACTIONS on INFORMATION SCIENCE and APPLICATIONS*, vol. 22, pp. 604–615, 2025.
- [23] D. Rothman, *RAG-Driven Generative AI: Build custom retrieval augmented generation pipelines with LlamaIndex, Deep Lake, and Pinecone*. Packt Publishing, 2024.
- [24] J. Bonifacino, R. Cadenas, T. Caetano, and S. Duclos, “Aura: Asistente universitario de recursos y aprendizaje,” Master’s thesis, Universidad ORT Uruguay, 2025.
- [25] N. Brenner, P. Hernández, I. Vallarino, and M. Wilner, “Chat your data: Chatbot de integración de datos con inteligencia artificial,” Master’s thesis, Universidad ORT Uruguay, 2023.
- [26] M. Harasymczuk, “2.3. fastapi architecture,” Webpage in *Python - from None to AI*, 2025, accessed: 2025-10-25. [Online]. Available: <https://python3.info/fastapi/fastapi/about-architecture.html>
- [27] M. Cantelon, M. Harter, T. Holowaychuk, and N. Rajlich, *Node.js in Action*. Manning Publications Co., 2014.
- [28] Asian Digital Hub, “Understanding the architecture of node.js applications,” 2024, accessed: 2025-10-25. [Online]. Available: <https://medium.com/@asiandigitalhub/understanding-the-architecture-of-node-js-applications-141eaa8c3ba6>
- [29] V. R. Vyshnavi and A. Malik, “Efficient way of web development using python and flask,” *International Journal of Recent Research Aspects*, vol. 6, no. 2, pp. 16–19, 2019.
- [30] D. Ghimire, “Comparative study on python web frameworks: Flask and django,” Master’s thesis, Metropolia University of Applied Sciences, 2020.
- [31] P. F. M. Calderón and M. G. Z. Mora, “Computación en la nube: la infraestructura como servicio frente al modelo on-premise,” *Dominio de las Ciencias*, vol. 6, no. 4, pp. 1535–1549, 2020.

- [32] cloudflare, “What is iaas (infrastructure-as-a-service)?” [Online]. Available: <https://www.cloudflare.com/learning/cloud/what-is-iaas/>
- [33] S. Mathew and J. Varia, “Overview of amazon web services,” *Amazon Whitepapers*, vol. 105, no. 1, p. 22, 2014.
- [34] Amazon, “Architecture overview - Workload Discovery on AWS.” [Online]. Available: <https://docs.aws.amazon.com/solutions/latest/workload-discovery-on-aws/architecture-overview.html>
- [35] B. C. Cárdenas Sánchez and C. A. Olarte Rojas, “Análisis de seguridad entre microservicios con amazon web service,” *Revista Logos Ciencia & Tecnología*, vol. 14, no. 2, pp. 42–52, 2022.
- [36] A. E. C. Cloud, “Amazon web services,” *Retrieved November*, vol. 9, no. 2011, p. 2011, 2011.
- [37] J. Qarkaxhija, “Using cloud computing as an infrastructure case study-microsoft azure,” 2020.
- [38] GeeksforGeeks, “Introduction to Microsoft Azure — A Cloud Computing Service,” 7 2025. [Online]. Available: <https://www.geeksforgeeks.org/devops/what-is-microsoft-azure/>
- [39] E. Páez, “Google cloud,” *DPL Cloud Digital Series*, 2022. [Online]. Available: <https://dplnews.com/wp-content/uploads/2022/10/dpl-cloud-Google-Cloud-bu%CC%81squeda-de-oportunidades-del-futuro.pdf>
- [40] O. Adwan, “An investigation of microsoft azure and amazon web services from users’ perspectives,” *International Journal of Emerging Technologies in Learning (iJET)*, 2019.
- [41] B. Gupta, P. Mittal, and T. Mufti, “A review on amazon web service (aws), microsoft azure & google cloud platform (gcp) services,” in *Proceedings of the 2nd International Conference on ICT for Digital, Smart, and Sustainable Development, ICIDSSD 2020, 27-28 February 2020, Jamia Hamdard, New Delhi, India*, 2021, p. 9.
- [42] U. Team, “Cloud vs On Premise Infrastructure : Choosing the Right Model,” 7 2025. [Online]. Available: <https://www.veritis.com/blog/cloud-vs-on-premise-it-infrastructure-model-of-your-choice/>
- [43] B. B. Rad, H. J. Bhatti, and M. Ahmadi, “An introduction to docker and analysis of its performance,” *International Journal of Computer Science and Network Security (IJCSNS)*, vol. 17, no. 3, p. 228, 2017.